

ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและอังกฤษแบบผสม

โดย

นายชิติพัทธ์ จำปา

โครงงานพิเศษนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2569
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและอังกฤษแบบผสม

โดย

นายชิติพัทธ์ จำปา

โครงงานพิเศษนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2569

ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

AI-Generated Text Detection and Classification System for Thai-English Code-switching

BY

Mr. Chitipat Jampa

A FINAL-YEAR PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF SCIENCE

COMPUTER SCIENCE

FACULTY OF SCIENCE AND TECHNOLOGY

THAMMASAT UNIVERSITY

ACADEMIC YEAR 2026

COPYRIGHT OF THAMMASAT UNIVERSITY

มหาวิทยาลัยธรรมศาสตร์
คณะวิทยาศาสตร์และเทคโนโลยี

รายงานโครงงานพิเศษ

ของ

นายชิติพัทธ์ จำปา

เรื่อง

ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและอังกฤษแบบผสม

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
เมื่อ วันที่ วันที่ ชื่อเต็มของเดือน พ.ศ. 2569

อาจารย์ที่ปรึกษา _____
(ผศ.ดร.ปกรณ์ ลี้สุทธิพรชัย)

กรรมการสอบโครงงานพิเศษ _____
(ผศ.ดร.วิลาวรรณ รักผกาวงศ์)

กรรมการสอบโครงงานพิเศษ _____
(อาจารย์สิริกันยา นิลพานิช)

หัวข้อโครงงานพิเศษ	ระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์สำหรับการใช้งานภาษาไทยและอังกฤษแบบผสม
ชื่อผู้เขียน	นายชิติพัทธ์ จำปา
ชื่อปริญญา	วิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
สาขาวิชา/คณะ/มหาวิทยาลัย	สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยธรรมศาสตร์
อาจารย์ที่ปรึกษาโครงงานพิเศษ	ผศ.ดร.ปกรณ์ ลี้สุทธิพรชัย
ปีการศึกษา	2569

บทคัดย่อ

โครงงานนี้พัฒนาระบบตรวจจับและจำแนกแหล่งที่มาของข้อความภาษาไทยเป็น 4 ประเภท ได้แก่ Human, GPT, Gemini และ Other โดยปรับแต่ง WangchanBERTa และประมวลผลภายในเว็บเบราว์เซอร์ผ่าน ONNX Runtime Web และ Web Worker แบบจำลองฉบับสมบูรณ์ใช้ W8A32 แบบ foldable weight-only ซึ่งเก็บน้ำหนักจำนวนเต็ม 8 บิตแบบรายช่องสัญญาณและคืนค่าน้ำหนักเพื่อคำนวณด้วย FP32 มีขนาด 106,299,948 ไบต์ และใช้ตัวตัดโทเคน Unigram/Metaspace พร้อมแผนผังปรับรูปอักขระที่บันทึกไว้และข้อกำหนดนำเข้า 400 โทเคนรวม BOS/EOS

ชุดข้อมูลข่าวภาษาไทยมี 38,428 ตัวอย่างจากบทความตั้งต้น 9,607 กลุ่มสมดุล 4 ประเภท แบ่งระดับกลุ่มเป็นชุดฝึกสอน 30,740 ตัวอย่าง ชุดทวนสอบ 3,844 ตัวอย่าง และชุดทดสอบ 3,844 ตัวอย่าง เลือก W8A32 ด้วยชุดทวนสอบซึ่งได้ Macro F1 ร้อยละ 88.8342 แล้วประเมินแบบจำลองที่ตรึงไว้บนชุดทดสอบเดิม ได้ Accuracy ร้อยละ 87.3569, Macro Precision ร้อยละ 88.8430, Macro Recall ร้อยละ 87.3569 และ Macro F1 ร้อยละ 87.3705 สูงกว่าเป้าหมายร้อยละ 85 ส่วน Human Precision เท่ากับร้อยละ 97.3624 และ Human Recall ร้อยละ 88.3455 จึงยังมีข้อความมนุษย์ถูกจำแนกเป็นปัญญาประดิษฐ์ 112 จาก 961 ตัวอย่าง หรือร้อยละ 11.6545 ผล FP32 อ้างอิงเดิมมี Macro F1 ร้อยละ 87.5074 ซึ่งเป็นคนละผลกับแบบจำลองฉบับสมบูรณ์ ทั้งนี้ชุดทดสอบเคยถูกตรวจระหว่างวินิจฉัยรุ่นก่อน การวิเคราะห์การปะปนอักษรไทย–ละตินเป็นเพียงการวิเคราะห์กลุ่มย่อยเชิงสำรวจ มิใช่เกณฑ์ทดสอบการสลับภาษาที่กำกับโดยผู้เชี่ยวชาญ การอนุมาน 36 ครั้งบน HTTPS Preview เป็นผลทดสอบเดิมแยกจากรุ่นเผยแพร่ปัจจุบัน ซึ่งเพิ่ม Supabase Auth/RLS, Edge Function และ Turnstile แล้ว ผลทดสอบตามกรณีที่กำหนดไม่ใช่การรับรองความปลอดภัยหรือความเข้ากันได้ทุกสภาพแวดล้อม

คำสำคัญ: การตรวจจับข้อความปัญญาประดิษฐ์, การระบุค่ายแบบจำลองภาษา, WangchanBERTa, การประมวลผลฝั่งผู้ใช้งาน, ONNX Runtime Web, INT8 Quantization, ความเป็นส่วนตัวของข้อมูล

Thesis Title	AI-Generated Text Detection and Classification System for Thai-English Code-switching
Author	Mr. Chitipat Jampa
Degree	Bachelor of Science
Major Field/Faculty/University	Computer Science Faculty of Science and Technology Thammasat University
Project Advisor	Asst.Prof.Dr. Pakorn Leesutthipornchai
Academic Year	2026

ABSTRACT

This project develops a four-class Thai text authorship classifier (Human, GPT, Gemini and Other) using fine-tuned WangchanBERTa and browser-side ONNX Runtime Web inference in a dedicated worker. The finalized foldable W8A32 model stores symmetric per-channel INT8 weights and reconstructs them for FP32 computation, without activation quantization. Its file size is 106,299,948 bytes. A pinned SentencePiece-style Unigram/Metaspace tokenizer executes the saved normalization map and preserves BOS/EOS within a 400-token input contract.

The balanced news corpus contains 38,428 examples from 9,607 article groups, partitioned into 30,740 training, 3,844 validation and 3,844 test examples. W8A32 was selected using validation data (88.8342% Macro F1). The fixed final model achieved 87.3569% accuracy, 88.8430% macro precision, 87.3569% macro recall and 87.3705% Macro F1 on the original test split, exceeding the 85% objective. Human precision was 97.3624%, while Human recall was 88.3455%: 112 of 961 human texts (11.6545%) were incorrectly classified as AI. Original FP32 test Macro F1 was 87.5074%. The test split had been inspected in earlier diagnostics and is not a never-inspected blind test. Article-group separation does not exclude semantic near-duplicates. Supplementary Thai/Latin script-mixing analysis is exploratory, not a linguistically annotated code-switching benchmark. An earlier protected inference-only Preview passed 36 functional inferences across Edge, Chrome and Firefox. The current release adds Supabase Auth, database-enforced administrator authorization, validated feedback ingestion and Turnstile. Functional and scoped security tests remain distinct from corpus-level accuracy evaluation and do not establish universal security or browser support.

Keywords: AI-Generated Text Detection, Vendor Attribution, WangchanBERTa, Client-side Inference, ONNX Runtime Web, INT8 Quantization, Data Privacy

กิตติกรรมประกาศ

โครงงานฉบับนี้สำเร็จลุล่วงไปได้ด้วยความกรุณาจากอาจารย์ปกรณ์ ลี้สุทธิพรชัย อาจารย์ที่ปรึกษาโครงงาน ที่ได้สละเวลาให้คำปรึกษา แนะนำแนวทาง และถ่ายทอดความรู้ที่เป็นประโยชน์อย่างยิ่งในการวางรากฐานโครงงาน ตลอดจนการเตรียมชุดข้อมูลและการวิเคราะห์ในเบื้องต้น ผู้วิจัยจึงขอขอบพระคุณมา ณ โอกาสนี้

นายชิติพัทธ์ จำปา

สารบัญ

หน้า

บทคัดย่อ 2

ABSTRACT 4

กิตติกรรมประกาศ 5

สารบัญ 6

สารบัญตาราง 10

สารบัญภาพ 11

รายการสัญลักษณ์และคำย่อ 12

บทที่ 1 บทนำ 1

1.1 ความเป็นมาและความสำคัญของโครงงาน 1

1.2 วัตถุประสงค์ 2

1.3 ขอบเขตของโครงงาน 3

1.4 ประโยชน์ของโครงงาน 5

1.5 ข้อจำกัดของโครงงาน 5

บทที่ 2 วรรณกรรมและงานวิจัยที่เกี่ยวข้อง 7

2.1 แนวคิดทฤษฎีที่เกี่ยวข้อง 7

2.1.1 แบบจำลอง WangchanBERTa และสถาปัตยกรรม RoBERTa/CamemBERT 7
2.1.2 หลักการตรวจจับข้อความที่สร้างโดยปัญญาประดิษฐ์และการระบุค่ายผู้ให้บริการ 8
2.1.3 การประมวลผลฝั่งผู้ใช้งาน WebAssembly และ ONNX Runtime Web 8
2.1.4 การบีบอัดแบบจำลอง (Model Quantization) และเทคโนโลยี ONNX Runtime Web 9
2.1.5 การป้องกันการรั่วไหลของข้อมูลด้วยการแบ่งข้อมูลระดับกลุ่ม (Group-level Splitting) 10
2.2 งานวิจัยที่เกี่ยวข้อง 11
2.2.1 การวิเคราะห์ประสิทธิภาพและข้อจำกัดของระบบตรวจจับในปัจจุบัน 11
บทที่ 3 วิธีการวิจัย 13
3.1 ภาพรวมของโครงงาน 13
3.1.1 ขั้นตอนการพัฒนาแบบจำลอง (Offline Development Phase) 13
3.1.2 ระบบให้บริการฝั่งผู้ใช้งาน (Client-side Serving Architecture) 13
3.1.3 แบบจำลองความเป็นส่วนตัวสองโหมด (Two-Mode Privacy Model) 13
3.2 การวิเคราะห์ขอบเขตและความต้องการของระบบ 14
3.2.1 ความต้องการด้านฟังก์ชันการทำงาน (Functional Requirements) 14
3.2.2 ความต้องการด้านคุณภาพและความปลอดภัย (Non-functional Requirements) 14
3.2.3 ความต้องการด้านสภาพแวดล้อมทางเทคนิค (Technical Requirements) 15
3.3 การดำเนินงาน 16
3.3.1 แหล่งข้อมูลมนุษย์และการลดอคติเชิงหัวข้อ (Topic De-biasing) 16
3.3.2 การขยายบทความและการควบคุมคำสั่งสร้างข้อความ (Per-article Expansion & Prompt Control) 16

3.3.3 การทำความสะอาดข้อมูล (Data Cleaning Filters) 17
3.3.4 ความสมดุลโดยการสร้าง (Balance by Construction) 18
3.3.5 การป้องกันข้อมูลรั่วไหลด้วยการแบ่งข้อมูลระดับกลุ่ม (GroupShuffleSplit) 18
3.3.6 การฝึกฝนแบบจำลอง (Model Fine-tuning) 19
3.3.7 การส่งออก บีบอัด และนำขึ้นใช้งาน (Export, Quantization & Deployment) 20
3.3.8 การวินิจฉัยความคลาดเคลื่อนและเลือก W8A32 20
3.3.9 ความเท่าเทียมของตัวตัดโทเคนและการประเมินตรึงขั้นสุดท้าย 21
บทที่ 4 ผลการดำเนินงาน 22
4.1 ผลการจัดเตรียมข้อมูล 22
4.2 ผลการทดสอบประสิทธิภาพแบบจำลอง 23
4.2.1 ตัวชี้วัดรวม (Aggregate Metrics) 23
4.2.2 เมทริกซ์ความสับสน (Confusion Matrix) 23
4.2.3 ตัวชี้วัดรายประเภท (Per-class Metrics) 24
4.2.4 การเกิดปรากฏการณ์แม่เหล็ก GPT (The GPT Magnet Effect) 24
4.2.5 นัยต่อการออกแบบส่วนแสดงผล 25
4.3 ผลการวินิจฉัยและเปรียบเทียบ quantization 25
4.4 ผลสำรวจการปะปนอักษรไทย–ละติน 25
4.5 ผลทดสอบเว็บแอปพลิเคชันและ HTTPS Preview 26
บทที่ 5 สรุป 29
5.1 สรุปผลการดำเนินงาน 29
5.2 อภิปรายผล 30
5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต 32
รายการอ้างอิง 33

ภาคผนวก 35

ภาคผนวก ก การเปิดเผยการใช้งาน Generative AI 36

ก.1 AI ที่ใช้สร้างชุดข้อมูลสังเคราะห์ (Synthetic Data Generation) 36

ก.2 AI ด้านการพัฒนาโปรแกรมและการตรวจสอบข้อผิดพลาด (Programming & Debugging) 36

ก.3 AI ด้านการสืบค้นวรรณกรรมและการจัดเตรียมเอกสาร (Literature Search & Documentation) 36

ก.4 คำรับรองความรับผิดชอบต่อผลงาน 37

ภาคผนวก ข สคริปต์การจัดการชุดข้อมูลและการแบ่งข้อมูลระดับกลุ่ม (Data Pipeline & Splitting) 38

ข.1 สคริปต์ทำความสะอาดและคัดกรองข้อมูล (Data Cleaning Filters) 38

ข.2 สคริปต์การแบ่งข้อมูลระดับกลุ่มบทความ (Group-level Splitting) 40

ข.3 คำสั่งกำเนิดข้อความสังเคราะห์ (Prompt Templates) 41

ภาคผนวก ค สคริปต์การแปลงและบีบอัดแบบจำลอง (Model Optimization & Quantization) 44

ค.1 สคริปต์การส่งออกกราฟการคำนวณ (ONNX Export) 44

ค.2 ประวัติสคริปต์ Dynamic INT8 ที่ไม่ใช่แบบจำลองสุดท้าย 45

ค.3 แบบจำลอง W8A32 ที่เลือกและหลักฐานทำซ้ำ 45

ภาคผนวก ง สคริปต์ระบบสนับสนุนและระบบอัตโนมัติ (System Infrastructure & Automation) 47

ง.1 เวิร์กโฟลว์ GitHub Actions สำหรับรักษาสถานะฐานข้อมูล 47

ง.2 หลักฐาน staging และขอบเขตที่ยังไม่รับรอง 48

สารบัญตาราง

หน้า

ตารางที่ 2.1 เปรียบเทียบและเหตุผลการเลือกโมเดล 7
ตารางที่ 2.2 การวิเคราะห์ช่องว่างทางงานวิจัย 11
ตารางที่ 3.1 โครงสร้างประเภทและแบบจำลองสร้างข้อความ 17
ตารางที่ 3.2 อภิมานพารามิเตอร์การฝึกสอนแบบจำลอง V3 20
ตารางที่ 4.1 สรุปชุดข้อมูล V3 หลังการแบ่งระดับกลุ่มบทความ 22
ตารางที่ 4.2 ตัวชี้วัดรวมบนชุดทวนสอบและชุดทดสอบ 23
ตารางที่ 4.3 เมทริกซ์ความสับสนบนชุดทดสอบ (row = ประเภทจริง, column = ประเภทที่ทำนาย; n = 961 ต่อประเภท) 23
ตารางที่ 4.4 ตัวชี้วัดรายประเภทบนชุดทดสอบ 24
ตารางที่ 4.5 การเลือก configuration ด้วย validation 3,844 ตัวอย่าง (MB ฐานสิบ) 25
ตารางที่ 4.6 คุณลักษณะข้อมูลแต่ละ split (ค่าเฉลี่ยรายเอกสาร) 26
ตารางที่ 4.7 ผลกลุ่ม script-mixing ของ test เดิมตามเอกสารเต็ม 26
ตารางที่ 4.8 ตัวแปรกวนและผลแยกตามโหมดของบทความต้นทาง 26
ตารางที่ 4.9 Local complete-app smoke: warm n=4 ต่อเบราว์เซอร์ 27
ตารางที่ 4.10 Protected HTTPS Preview: warm n=4 ต่อเบราว์เซอร์ 27
ตารางที่ 4.11 ความสมบูรณ์ ความทนทาน และการแยก staging 28

สารบัญภาพ

หน้า

ภาพที่ 2.1 การจัดเก็บน้ำหนักแบบ INT8
รายช่องสัญญาณและการคืนค่าน้ำหนักเพื่อคำนวณแบบ FP32 ของแบบจำลอง
W8A32 ที่เลือก (ภาพจัดทำโดยผู้จัดทำจากกระบวนการทดลองของโครงงาน) 10

รายการสัญลักษณ์และคำย่อ

สัญลักษณ์/คำย่อ	คำเต็ม/คำจำกัดความ
AI	Artificial Intelligence
API	Application Programming Interface
W8A32	INT8 weight storage with FP32 computation
CS	Code-switching
FP32	32-bit Floating Point
GPU	Graphics Processing Unit
INT8	8-bit Integer
LLM	Large Language Model
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
OOD	Out-of-Distribution
PDPA	Personal Data Protection Act
RLHF	Reinforcement Learning from Human Feedback
RLS	Row Level Security
SIMD	Single Instruction, Multiple Data
UI	User Interface
UX	User Experience
VRAM	Video Random Access Memory
WASM	WebAssembly
Client-side inference	ประมวลผลภายในอุปกรณ์ตามขอบเขตทดสอบ ไม่รับรองระบบทุกส่วน

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

ในปัจจุบัน แบบจำลองภาษาขนาดใหญ่ (Large Language Models: LLMs) เช่น GPT ของ OpenAI, Gemini ของ Google ตลอดจนแบบจำลองแบบเปิดเผยค่าน้ำหนัก (open-weight models) เช่น DeepSeek และ Qwen มีความสามารถในการสร้างข้อความภาษาไทยที่แนบเนียนใกล้เคียงงานเขียนของมนุษย์ในระดับที่ผู้อ่านทั่วไปแยกแยะได้ยาก ความสามารถดังกล่าวก่อให้เกิดความท้าทายเชิงสังคมหลายประการ ทั้งด้านความซื่อสัตย์ทางวิชาการ (academic integrity) ความน่าเชื่อถือของข่าวสาร และความโปร่งใสของเนื้อหาดิจิทัล สังคมจึงมีความต้องการเครื่องมือที่ช่วยตรวจสอบแหล่งที่มาของข้อความภาษาไทยที่เชื่อถือได้

ระบบตรวจจับที่ประมวลผลฝั่งเซิร์ฟเวอร์ต้องส่งข้อความออกจากอุปกรณ์ผู้ใช้ จึงมีข้อพิจารณาด้านความเป็นส่วนตัวและนโยบายจัดเก็บข้อมูล โครงงานนี้ศึกษาทางเลือกการประมวลผลภายในเบราว์เซอร์สำหรับข่าวภาษาไทย โดยไม่สรุปว่าเครื่องมือตรวจจับทั้งหมดใช้สถาปัตยกรรมเดียวกันหรือให้ผลเทียบเคียงกันข้ามภาษาได้

โครงงานพิเศษ 1 (SP1) ได้วางรากฐานเพื่อตอบข้อจำกัดทั้งสอง โดยศึกษาความเป็นไปได้ของการจำแนกข้อความแบบสองประเภท (มนุษย์/ปัญญาประดิษฐ์) ด้วยแบบจำลอง WangchanBERTa บนชุดข้อมูลนำร่องที่รวบรวมจากกระทู้ Pantip และข้อความสังเคราะห์จากแบบจำลอง Llama 3 พร้อมทั้งพัฒนาส่วนติดต่อผู้ใช้งานต้นแบบและออกแบบแนวคิดสถาปัตยกรรมการประมวลผลฝั่งผู้ใช้งานไว้ ผลการศึกษาระยะแรกยืนยันความเป็นไปได้เชิงเทคนิค แต่ก็เผยให้เห็นข้อจำกัดเชิงระเบียบวิธีหลายประการ ทั้งขนาดชุดข้อมูลที่เล็ก (670 ตัวอย่าง) ความครอบคลุมของค่ายแบบจำลองที่จำกัดเพียงค่ายเดียว และความเสี่ยงที่ตัวจำแนกจะเรียนรู้ "ทางลัด" เชิงหัวข้อหรือเชิงเนื้อหาแทนการเรียนรู้สไตล์การประพันธ์ที่แท้จริง

โครงงานพิเศษ 2 (SP2) ฉบับนี้จึงยกระดับงานจากระยะแรกอย่างเป็นระบบภายใต้หลักการออกแบบสามประการ ได้แก่

ประการที่หนึ่ง ขยายโจทย์สู่การจำแนก 4 ประเภท ได้แก่ Human, GPT (gpt-4o-mini), Gemini (gemini-2.5-flash-lite) และ Other (deepseek/deepseek-chat และ qwen/qwen-2.5-7b-instruct) บนคลังข่าวสมดุล 38,428 ตัวอย่าง เป็นกรณีศึกษาการระบุแหล่งกำเนิดข้อความภาษาไทยภายใต้ขอบเขตผู้สร้างและแหล่งข่าวที่กำหนด มิได้อ้างความเป็นงานแรกของภาษาไทย

ประการที่สอง ประมวลผลภายในเบราว์เซอร์ด้วย ONNX Runtime Web/WASM และ Web Worker แบบจำลอง W8A32 ขนาด 106,299,948 ไบต์ ตัวตัดโทเคนและรันไทม์ถูกจัดชุดไว้ที่เส้นทางมีรุ่นกำกับบนต้นทางเดียวกับเว็บบน Vercel โดยไม่เปลี่ยนแบบจำลอง R2 เดิม การอนุมานไม่เรียกเซิร์ฟเวอร์ประมวลผลข้อความของโครงงาน ส่วนการส่งข้อมูลป้อนกลับเป็นการส่งข้อความและผลทำนายโดยสมัครใจ แยกจากการตรวจข้อความตามปกติ

ประการที่สาม ลดความจำเป็นของเซิร์ฟเวอร์อนุมานและ GPU ฝั่งผู้ให้บริการโดยย้ายภาระคำนวณสู่อุปกรณ์ผู้ใช้ อย่างไรก็ดี การเก็บไฟล์ การดาวน์โหลด แบนด์วิดท์ โควตาโฮสติ้ง และทรัพยากรผู้ใช้ยังมีต้นทุนและข้อจำกัด จึงไม่รับประกันต้นทุนศูนย์หรือรองรับผู้ใช้ได้ไม่จำกัด

แบบจำลองฉบับสมบูรณ์ผ่านการประเมินชุดทดสอบเดิม 3,844 ตัวอย่าง ได้ Macro F1 ร้อยละ 87.3705 และ Human Precision ร้อยละ 97.3624 โดย Human Recall เท่ากับร้อยละ 88.3455 จึงต้องพิจารณาความเสี่ยงกล่าวหางานเขียนมนุษย์ร่วมด้วย ผลทดสอบ inference-only Preview เป็นหลักฐานระยะก่อนหน้า ส่วนรุ่นเผยแพร่ปัจจุบันเพิ่มการพิสูจน์ตัวตน การอนุญาตผู้ดูแล และข้อมูลป้อนกลับที่ตรวจสอบฝั่งบริการแล้ว โดยไม่เปลี่ยนแบบจำลองที่ประเมิน

1.2 วัตถุประสงค์

1. เพื่อพัฒนาแบบจำลองจำแนกข้อความภาษาไทยแบบ 4 ประเภท (มนุษย์ / GPT / Gemini / แบบจำลองเปิดเผยค่าน้ำหนักค่ายอื่น) บนโครงสร้าง WangchanBERTa โดยมีค่า Macro F1 บนชุดข้อมูลทดสอบไม่ต่ำกว่าร้อยละ 85 และค่า Precision ของประเภทมนุษย์ไม่ต่ำกว่าร้อยละ 95
2. เพื่อสร้างคลังข้อมูลภาษาไทยสมดุล 4 ประเภท โดยลดตัวแปรกวนด้วยแหล่งข่าวสองแห่ง ใช้แม่แบบคำสั่งส่วนกลาง และแบ่งระดับกลุ่มบทความประมาณ 80/10/10 พร้อมตรวจสอบการไม่ข้ามชุดของรหัสกลุ่ม ทั้งนี้ยังต้องเปิดเผยตัวแปรกวนและความเสี่ยงข้อความใกล้เคียงที่วิธีดังกล่าวไม่ได้ขจัด
3. เพื่อพัฒนาการอนุมานฝั่งผู้ใช้ด้วย ONNX Runtime Web และ W8A32 พร้อมตรึงความเข้ากันได้ของแบบจำลอง–ตัวตัดโทเคน ตรวจสอบความสมบูรณ์ของไฟล์และการกู้คืนเมื่อเริ่มระบบล้มเหลว เพื่อลดการส่งข้อความและภาระคำนวณฝั่งบริการภายใต้ข้อจำกัดที่ตรวจสอบได้

4. เพื่อประเมินประสิทธิภาพของระบบอย่างรอบด้านด้วยเมทริกซ์ความสับสน (confusion matrix) และตัวชี้วัดรายประเภท พร้อมวิเคราะห์รูปแบบความผิดพลาดเชิงความหมาย (semantic error analysis) เพื่อเป็นองค์ความรู้สำหรับงานวิจัยด้านการระบุค่ายแบบจำลองภาษาในภาษาไทยต่อไป

1.3 ขอบเขตของโครงงาน

เพื่อให้การดำเนินงานเป็นไปตามวัตถุประสงค์ที่กำหนดไว้ ผู้จัดทำได้กำหนดขอบเขตของโครงงานไว้ดังนี้:

1. **ด้านโมเดลและการตรวจจับ (Model & Detection):**
    - ใช้แบบจำลองภาษา WangchanBERTa เป็นโครงสร้างหลักในการวิเคราะห์และจำแนกข้อความ โดยปรับแต่งด้วยหัวจำแนก (classification head) แบบ 4 ประเภท
    - จำแนกแหล่งที่มาของข้อความออกเป็น 4 ประเภทที่สมดุลสมบูรณ์ ได้แก่

      -ประเภท 0 มนุษย์ (Human): บทความข่าวที่เขียนโดยนักข่าวจากสำนักข่าว Sanook และ The Standard

      -ประเภท 1 GPT: ข้อความที่สร้างโดยแบบจำลอง `gpt-4o-mini` ของ OpenAI

      -ประเภท 2 Gemini: ข้อความที่สร้างโดยแบบจำลอง `gemini-2.5-flash-lite` ของ Google

      – ประเภท 3 Other: ข้อความจาก deepseek/deepseek-chat และ qwen/qwen-2.5-7b-instruct ผ่าน OpenRouter มีเป้าหมายก่อนกรอง 5,000 ตัวอย่างต่อแบบจำลอง หลังเก็บเฉพาะกลุ่มครบสี่ประเภทจะเหลือ Other รวม 9,607 ตัวอย่าง ไม่ใช่ตัวแทนแบบจำลองที่ไม่รู้จักทั้งหมด
    - คลังข้อมูลฝึกสอนครอบคลุมข้อความภาษาไทยทำเนียบข่าว โดยทุกตัวอย่างผ่านตัวกรองสัดส่วนอักขระไทยไม่ต่ำกว่า 0.5 เพื่อป้องกันมิให้แบบจำลองเรียนรู้ "สัญญาณภาษา" แทนสไตล์การประพันธ์
    - ชุดข้อมูลสุดท้ายมีขนาด 38,428 แถว จากบทความตั้งต้น 9,607 บทความ สมดุลทั้ง 4 ประเภทโดยการสร้าง (balanced by construction)
2. **ด้านสถาปัตยกรรมและการประมวลผล (Architecture & Processing):**
    - ระบบเป็นเว็บแอปพลิเคชัน (Vite + React 19 + TypeScript) ที่ประมวลผลการตรวจจับบนเบราว์เซอร์ของผู้ใช้เท่านั้น ผ่าน ONNX Runtime Web (WebAssembly execution provider) ภายใน Web Worker เฉพาะกิจ

- แบบจำลองที่ใช้ในระบบฉบับสมบูรณ์คือ W8A32 แบบ foldable per-channel INT8 weight storage ขนาด 106,299,948 ไบต์ มิใช่ไฟล์ dynamic INT8 เดิม เก็บที่ /artifacts/thai-en-w8a32-4baa5cb331f6-v1/model.onnx พร้อม manifest และตัวตัดโทเคน SHA256 ตรงกับการทดลองที่อนุมัติ และตรวจสอบแคชก่อนใช้
- การอนุมานไม่มีเซิร์ฟเวอร์ประมวลผลข้อความของโครงงาน ใช้ไฟล์สถิตบน Vercel และประมวลผลใน Web Worker ส่วน Supabase Auth, ฐานข้อมูล และ Edge Function รองรับเฉพาะผู้ดูแลและข้อมูลป้อนกลับ การขัดข้องของบริการเสริมไม่ควรหยุดการตรวจข้อความ
- ในการทดสอบ Preview แบบ inference-only ไม่พบคำขอ Supabase หรือการส่งข้อมูลป้อนกลับ การตรวจพบดังกล่าวมีขอบเขตตามกรณีทดสอบ ไม่ใช่การรับรองความปลอดภัยสมบูรณ์

3. **ด้านฟังก์ชันการทำงาน (Features):**
   - รับข้อความผ่านการพิมพ์หรือวางโดยตรง ความยาว 200–10,000 ตัวอักษร
   - แสดงผลคำตัดสิน (verdict) พร้อมมาตรวัดความเชื่อมั่นของประเภทที่ชนะ และแผนภูมิแท่งแสดงการแจกแจงความน่าจะเป็นครบทั้ง 4 ประเภท (softmax distribution) เพื่อความโปร่งใสของผลวิเคราะห์
   - ข้อมูลป้อนกลับเป็นฟังก์ชันสมัครใจ ต้องยินยอมส่งข้อความ ป้ายกำกับที่ผู้ใช้คาดหวัง และผลทำนาย ผ่าน Turnstile และ Edge Function ซึ่งตรวจข้อมูลก่อนเรียกฟังก์ชันบันทึกฝั่งบริการ ผู้ใช้ทั่วไปไม่มีสิทธิ์ INSERT โดยตรงใน user_reports และสถานะเริ่มต้นถูกบังคับเป็น pending
   - ผู้ดูแลเข้าสู่ระบบด้วย Supabase Auth และต้องอยู่ใน allowlist ที่แก้ไขได้ เฉพาะผู้ดำเนินงานที่มีสิทธิ์ ฐานข้อมูลตรวจ session และสิทธิ์ผ่าน RLS ก่อนอ่าน เปลี่ยนสถานะ หรือลบรายงาน ผู้ใช้ที่เข้าสู่ระบบทั่วไปไม่ได้เป็นผู้ดูแลโดยอัตโนมัติ และไม่มีรหัสผ่านผู้ดูแลฝังใน JavaScript
4. **ขอบเขตที่ไม่ครอบคลุม:**
   - ไม่มีฟังก์ชันสกัดข้อความจากรูปภาพ (OCR) ฟังก์ชันนี้ซึ่งเคยอยู่ในแผนระยะ SP1 ถูกถอดออกตามคำแนะนำของอาจารย์ที่ปรึกษา เพื่อให้โครงงานมุ่งเน้นแก่นของปัญหาการจำแนกข้อความ
   - ไม่ครอบคลุมการตรวจจับแบบจำลองภาษาค่ายอื่นนอกเหนือจากที่อยู่ในชุดข้อมูลฝึกสอน (เช่น Claude, Llama, Typhoon) อย่างเป็นทางการ ดูข้อจำกัดในหัวข้อ 1.5

1.4 ประโยชน์ของโครงงาน

จากการดำเนินโครงงานพัฒนาระบบตรวจจับและจำแนกข้อความที่สร้างโดยปัญญาประดิษฐ์ ผู้จัดทำคาดว่าจะได้รับประโยชน์ดังต่อไปนี้:

1. **ด้านเครื่องมือและการตรวจสอบ (Tools & Verification):**
    - ได้เครื่องมือทดลองประกอบการพิจารณาแหล่งที่มาของข่าวภาษาไทย โดย Human Precision ร้อยละ 97.3624 อธิบายความถูกต้องในกลุ่มที่ทำนายเป็น Human ส่วน Human Recall ร้อยละ 88.3455 แสดงว่ายังมีความเสี่ยงกล่าวหามนุษย์ผิดร้อยละ 11.6545 ในชุดทดสอบนี้ จึงไม่ควรใช้ผลเป็นหลักฐานตัดสินทางวินัยเพียงอย่างเดียว
2. **ด้านความเป็นส่วนตัวและความปลอดภัย (Privacy & Security):**
    - ได้กรณีศึกษาการลดการส่งข้อความด้วยการอนุมานภายในเบราว์เซอร์ พร้อมแยกข้อมูลป้อนกลับที่ต้องขอความยินยอมและบังคับสิทธิ์ฝั่งบริการ การออกแบบและผลทดสอบเฉพาะกรณีไม่ใช่การรับรองการปฏิบัติตามกฎหมายหรือความปลอดภัยครบถ้วน
3. **ด้านองค์ความรู้ทางวิศวกรรม (Engineering Knowledge):**
    - ได้กรณีศึกษาตั้งแต่ fine-tuning, ONNX export, การวินิจฉัยความคลาดเคลื่อนของ quantization จนถึง W8A32 ซึ่งลดขนาดดาวน์โหลดประมาณร้อยละ 74.76 จาก FP32 ประมาณ 421.21 MB โดยคงการคำนวณ FP32 พัฒนา Web Worker การดาวน์โหลดที่ตรวจสอบ SHA256 แคชทางเลือกและการกู้คืนที่มีเวลาจำกัด ไม่อ้างการลด RAM ร้อยละ 75 หรือต้นทุนให้บริการศูนย์
4. **ด้านการพัฒนาอย่างต่อเนื่อง (Sustainability & Improvement):**
    - ข้อมูลป้อนกลับเป็นความเห็นของผู้ใช้ แม้มีการยินยอมและควบคุมสิทธิ์แล้วก็ไม่ใช่หลักฐานผู้ประพันธ์ที่ยืนยันอิสระ และไม่ถูกนำเข้าชุดฝึกหรือชุดทดสอบโดยอัตโนมัติ การนำไปวิจัยภายหลังต้องตรวจ provenance ป้ายกำกับ และสิทธิ์การใช้เพิ่มเติม

1.5 ข้อจำกัดของโครงงาน

1. **ด้านทรัพยากรของอุปกรณ์ผู้ใช้งาน (Hardware Limitations):**
    - ประสิทธิภาพและหน่วยความจำแปรตามอุปกรณ์ การทดสอบ HTTPS บนคอมพิวเตอร์ i7-13700HX/RAM ประมาณ 32 GiB พบค่ามัธยฐาน warm worker round-trip 1.684–1.772 วินาที และ sampled peak private commit รวมตระกูลกระบวนการเบราว์เซอร์ 1.962–2.449 GiB ช่วงเริ่มระบบ/กู้คืน ค่านี้ไม่ใช่ WASM heap อย่างเดียวหรือเกณฑ์ RAM ขั้นต่ำ และอาจพลาดยอด

ระยะสั้น การทดลองก่อนหน้าพบประมาณ 3.55 GiB จึงยังไม่รับรองมือถือ Safari อุปกรณ์หน่วยความจำน้อย หรือเบราว์เซอร์รุ่นเก่า

2. **ด้านการดาวน์โหลดครั้งแรก (First-load Cost):**
    - ครั้งแรกต้องดาวน์โหลดแบบจำลอง 106.30 MB รวมทั้งไฟล์รันไทม์และตัวตัดโทเคน ชุด staging รวม 161,953,395 ไบต์ การเริ่มระบบใหม่ในเบราว์เซอร์ที่ไม่มีแคชบน HTTPS ใช้ประมาณ 5.516–6.344 วินาทีในเงื่อนไขที่ CDN อุ่นแล้ว เมื่อโหลดซ้ำด้วยแคชที่ตรวจสอบได้ใช้ประมาณ 1.265–1.330 วินาที Cache API อาจถูกปฏิเสธหรือถูกล้างได้ และการมีแคชแบบจำลองไม่ยืนยันการเริ่มเว็บไซต์แบบออฟไลน์ครบชุด
3. **ด้านลักษณะข้อความนำเข้า (Input Text Characteristics):**
    - แบบจำลองถูกฝึกฝนบนข้อความที่มีสัดส่วนอักขระไทยไม่ต่ำกว่า 0.5 (ร้อยละ 50) ข้อความนำเข้าจึงควรเป็นภาษาไทยเป็นหลัก ข้อความที่เป็นภาษาอังกฤษล้วนหรือมีสัดส่วนภาษาอื่นเกินครึ่ง อยู่นอกการแจกแจงของข้อมูลฝึกสอนและให้ผลที่เชื่อถือไม่ได้ นอกจากนี้ระบบกำหนดความยาวข้อความขั้นต่ำ 200 ตัวอักษร (เพื่อให้มีบริบทเชิงสไตล์เพียงพอ) และสูงสุด 10,000 ตัวอักษร
4. **ด้านแบบจำลองนอกการแจกแจง (Out-of-Distribution Vendors):**
    - ประเภท Other แทนแหล่งกำเนิด DeepSeek/Qwen ที่มีในข้อมูลเท่านั้น ระบบ argmax บังคับเลือกหนึ่งในสี่ประเภทและไม่มีกลไกตรวจพบค่ายใหม่ ประสิทธิภาพต่อ Claude, Llama, Typhoon หรือผู้สร้างรุ่นอื่นยังไม่ได้รับการประเมิน จึงไม่ควรคาดเดาว่าจะถูกจัดเป็นประเภทใด
5. **ด้านทำเนียบภาษาของข้อมูล (Domain Shift):**
    - คลังข้อมูลฝึกสอนเป็นข้อความทำเนียบข่าว (Sanook และ The Standard) ประสิทธิภาพบนข้อความประเภทอื่น เช่น เรียงความ ข้อความสนทนาบนสื่อสังคมออนไลน์ หรือบทความวิชาการ ยังไม่ได้รับการประเมินอย่างเป็นระบบ
6. **ด้านความปลอดภัยของระบบส่วนเสริม:**
    - รุ่นปัจจุบันแก้ข้อจำกัดเดิมด้วย Supabase Auth, allowlist/RLS และการจำกัดอัตราฝั่งบริการแล้ว แต่การทดสอบตามกรณีที่บันทึกไว้ไม่ใช่ penetration test ครบวงจร ยังไม่ได้บังคับ MFA หรือทดสอบกรณี CAPTCHA และ session expiry ทุกชนิด การเก็บ token ในหน่วยความจำลดการคงอยู่ข้ามการโหลดหน้า แต่ไม่ทำให้ปลอดภัยจาก XSS โดยสมบูรณ์

บทที่ 2
วรรณกรรมและงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดทฤษฎีที่เกี่ยวข้อง

2.1.1 แบบจำลอง WangchanBERTa และสถาปัตยกรรม RoBERTa/CamemBERT

WangchanBERTa (Lowphansirikul et al., 2021) เป็นแบบจำลองภาษาที่ผ่านการฝึกล่วงหน้า (pre-trained language model) สำหรับภาษาไทย สร้างบนสถาปัตยกรรม RoBERTa (Liu et al., 2019) ในรูปแบบ CamemBERT (Martin et al., 2020) รุ่นที่ใช้ในโครงงานนี้คือ `airesearch/wangchanberta-base-att-spm-uncased` ซึ่งมีคุณลักษณะทางสถาปัตยกรรมดังนี้: ตัวเข้ารหัส Transformer จำนวน 12 ชั้น, มิติเวกเตอร์ซ่อน (hidden size) 768, หัวกลไกความสนใจ (attention heads) 12 หัว, มิติชั้น feed-forward (intermediate size) 3,072, คลังคำศัพท์แบบ SentencePiece ขนาด 25,005 หน่วยคำย่อย (subword) และตำแหน่งสูงสุด 512 โทเคน รวมพารามิเตอร์ประมาณ 106 ล้านตัว

การเปรียบเทียบและเหตุผลการเลือกโมเดลในรูปแบบตาราง

ตัวตัดโทเคนที่บันทึกกับ checkpoint เป็น SentencePiece-style Unigram ร่วมกับ Metaspace และ Sequence normalizer ที่มี Precompiled charsmap, Strip และ Replace ไม่ใช่ Byte-level BPE การเลือกฐาน WangchanBERTa อาศัยความเหมาะสมต่อภาษาไทย แต่โครงงานนี้ไม่ได้เปรียบเทียบ mBERT/XLM-R ภายใต้ชุดทดลองเดียวกัน

ตารางที่ 2.1 เปรียบเทียบและเหตุผลการเลือกโมเดล

คุณสมบัติ	**mBERT / XLM-R**	**WangchanBERTa ที่ใช้**
ข้อมูลก่อนฝึก	หลายภาษา	เน้นภาษาไทย
Tokenizer	WordPiece / SentencePiece	Unigram + Metaspace (saved charsmap)
ขอบเขตการเปรียบเทียบ	ไม่ได้ทดสอบด้วย protocol เดียวกันในโครงงาน	เลือกเป็นฐาน; ไม่ใช่อันดับเหนือกว่าทุกงาน

ตารางที่ 2.1 เป็นการเปรียบเทียบลักษณะโครงสร้าง มิใช่อันดับผลทดลอง ตัวตัดโทเคน Unigram รองรับข้อความที่พบในข้อมูล แต่การรองรับอักขระผสมมิได้ยืนยันความแม่นยำต่อการสลับภาษาเชิงภาษาศาสตร์ ซึ่งยังขาดการกำกับช่วงภาษาและการทดสอบเฉพาะ

ในการปรับแต่งสำหรับโจทย์ของโครงงาน หัวจำแนกถูกกำหนดเป็น CamembertForSequenceClassification ที่มี num_labels = 4 พร้อมการจับคู่ป้ายกำกับ {0: Human, 1: GPT, 2: Gemini, 3: Other} การจำแนกหลายประเภทใช้ฟังก์ชัน softmax บนเวกเตอร์ logits ขนาด 4 มิติ และฟังก์ชันสูญเสียแบบ categorical cross-entropy

2.1.2 หลักการตรวจจับข้อความที่สร้างโดยปัญญาประดิษฐ์และการระบุค่ายผู้ให้บริการ

การตรวจจับข้อความปัญญาประดิษฐ์ตั้งอยู่บนสมมติฐานว่าแบบจำลองภาษาทิ้ง "ลายนิ้วมือเชิงสไตล์" (stylistic fingerprint) ไว้ในข้อความที่สร้าง เช่น จังหวะการวางย่อหน้าที่สม่ำเสมอเกินธรรมชาติ การเลือกใช้คำเชื่อมแบบระมัดระวัง โครงสร้างเปิด-สรุปที่เป็นแบบแผน และการแจกแจงคำศัพท์ที่มีเอนโทรปีต่ำกว่างานเขียนมนุษย์ แนวทางการตรวจจับแบ่งได้สองสายหลัก ได้แก่ (1) แนวทาง zero-shot ที่อาศัยคุณสมบัติเชิงความน่าจะเป็นของข้อความภายใต้แบบจำลองภาษา เช่น DetectGPT (Mitchell et al., 2023) ซึ่งใช้ความโค้งของ log-likelihood แต่มีต้นทุนการคำนวณสูงและผูกกับแบบจำลองอ้างอิง และ (2) แนวทาง supervised fine-tuning ที่ฝึกฝนตัวจำแนกบนคู่ตัวอย่างมนุษย์–ปัญญาประดิษฐ์โดยตรง ซึ่งให้ความแม่นยำสูงกว่าในโดเมนที่มีข้อมูลฝึกสอนเพียงพอ

โครงงานนี้เลือกแนวทางที่สองด้วยเหตุผลสองประการ ประการแรก ระบบต้องการแบบจำลองขนาดกะทัดรัดที่ทำงานบนเบราว์เซอร์ได้จริง ซึ่งแนวทาง zero-shot ที่ต้องเรียกใช้แบบจำลองภาษาอ้างอิงขนาดใหญ่ไม่สามารถตอบโจทย์ได้ ประการที่สอง โครงงานต้องการขยายจากการจำแนกสองประเภทไปสู่การระบุค่าย (vendor attribution) ซึ่งเป็นโจทย์ multi-class โดยธรรมชาติ และแนวทาง supervised รองรับโดยตรงเพียงเปลี่ยนมิติของหัวจำแนก

ความสับสนระหว่างค่ายอาจเกิดจากรูปแบบข้อความที่ทับซ้อนกัน แต่ข้อมูลการจำแนกเพียงอย่างเดียวไม่ระบุสาเหตุจาก distillation หรือ RLHF ของผู้สร้าง โครงงานจึงใช้คำว่า GPT magnet effect เป็นชื่อบรรยายรูปแบบในเมทริกซ์ความสับสน ไม่ใช่ข้อพิสูจน์กลไกการฝึกหรือความเป็นงานค้นพบแรก

2.1.3 การประมวลผลฝั่งผู้ใช้งาน WebAssembly และ ONNX Runtime Web

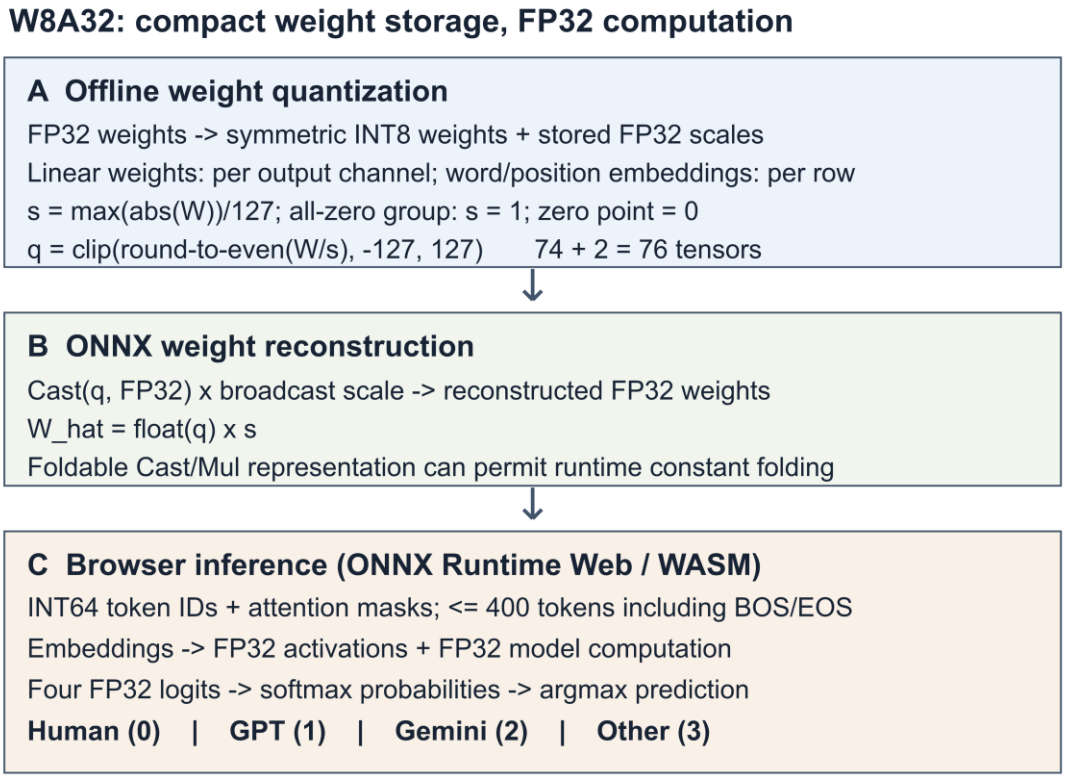
โดย WebAssembly (WASM) เป็นรูปแบบคำสั่งระดับต่ำที่ทำงานในเบราว์เซอร์ด้วยความเร็วใกล้เคียงโค้ดเนทีฟ ส่วนขยาย WASM SIMD (Single Instruction, Multiple Data) เปิดให้คำสั่งเวกเตอร์ 128 บิตของเบราว์เซอร์ถูกแปลงลงสู่ชุดคำสั่งเวกเตอร์ของหน่วยประมวลผลจริง (เช่น AVX2 บนสถาปัตยกรรม x86-64 หรือ NEON บน ARM64) ซึ่งจำเป็น

อย่างยิ่งต่อการคูณเมทริกซ์อันเป็นภาระงานหลักของสถาปัตยกรรม Transformer ด้วยเหตุนี้ การรองรับ WASM SIMD ของเบราว์เซอร์จึงเป็นความต้องการขั้นต่ำของระบบ

ONNX (Open Neural Network Exchange) เป็นรูปแบบกลางสำหรับแลกเปลี่ยนแบบจำลองข้ามกรอบงาน และ ONNX Runtime Web เป็นรันไทม์ของ Microsoft ที่ประมวลผลกราฟ ONNX ในเบราว์เซอร์ผ่าน execution provider แบบ WASM ในโครงงานนี้ การประมวลผลทั้งหมดเกิดใน Web Worker เฉพาะกิจเพื่อมิให้การคำนวณหนักปิดกั้นเธรดหลักของหน้าเว็บ การออกแบบนี้ทำให้สถาปัตยกรรม client-side intelligence เป็นไปได้จริงโดยผู้ใช้ไม่ต้องติดตั้งซอฟต์แวร์ใด ๆ เพิ่มเติม

2.1.4 การบีบอัดแบบจำลอง (Model Quantization) และเทคโนโลยี ONNX Runtime Web

Quantization ลดความละเอียดของการเก็บค่าน้ำหนักหรือการคำนวณ ซึ่งต้องแยกจากกัน รุ่น dynamic INT8 เดิมใช้ per-tensor scaling และ quantize activations ขณะทำงาน พบการจำแนกเสื่อมลงมาก รุ่นที่เลือกจึงเป็น W8A32: น้ำหนัก linear ใช้ symmetric INT8 แบบ per-output-channel, word/position embeddings แบบ per-row และคง token-type embeddings, biases และ normalization เป็น FP32 ใช้ Cast/Mul คืนค่าน้ำหนักที่ constant folding ได้ก่อนการคำนวณ FP32 จึงลดไฟล์ดาวน์โหลดประมาณร้อยละ 74.76 แต่ไม่ใช่การประมวลผลจำนวนเต็มทั้งระบบหรือการลดหน่วยความจำตามสัดส่วนไฟล์



ภาพที่ 2.1 การจัดเก็บน้ำหนักแบบ INT8 รายช่องสัญญาณและการคืนค่าน้ำหนักเพื่อคำนวณแบบ FP32 ของแบบจำลอง W8A32 ที่เลือก (ภาพจัดทำโดยผู้จัดทำจากกระบวนการทดลองของโครงงาน)

ภาพที่ 2.1 จัดทำขึ้นเพื่ออธิบายวิธีของโครงงาน แบ่งเป็นการเก็บน้ำหนัก INT8 และสเกล FP32 แบบรายช่องสัญญาณหรือรายแถว การคืนค่าน้ำหนักด้วย Cast/Mul ซึ่งเปิดให้รันไทม์ทำ constant folding และการอนุมานด้วย activations และการคำนวณ FP32 ข้อมูลเข้าเป็น token IDs และ attention masks ชนิด INT64 ภาพนี้เป็นแผนภาพกระบวนการ ไม่ใช่กราฟ ONNX จริงหรือผลวัดประสิทธิภาพ และไม่แสดงการ quantize activations

2.1.5 การป้องกันการรั่วไหลของข้อมูลด้วยการแบ่งข้อมูลระดับกลุ่ม (Group-level Splitting)

ในชุดข้อมูลที่หลายแถวสืบทอดมาจากต้นทางเดียวกัน (ในโครงงานนี้: บทความมนุษย์หนึ่งบทความถูกขยายเป็น 4 แถว ต้นฉบับมนุษย์และฉบับที่ปัญญาประดิษฐ์ 3 ค่ายสร้างจากบทความเดียวกัน) การสุ่มแบ่งข้อมูลระดับแถว (row-level random split) จะทำให้แถวที่มีเนื้อหาร่วมกันกระจายไปอยู่ทั้งชุดฝึกสอนและชุดทดสอบ แบบจำลองจึงสามารถ "จำเนื้อหา" ของบทความแทนการเรียนรู้สไตล์การประพันธ์ ส่งผลให้ตัวเลขประเมินสูงเกินจริง วิธี

ป้องกันมาตรฐานคือการแบ่งระดับกลุ่ม โดยใช้รหัสบทความเป็นหน่วยการแบ่ง รายละเอียดการประยุกต์ใช้ในโครงงานนี้อยู่ในหัวข้อ 3.3.5

2.2 งานวิจัยที่เกี่ยวข้อง

2.2.1 การวิเคราะห์ประสิทธิภาพและข้อจำกัดของระบบตรวจจับในปัจจุบัน

งานที่เกี่ยวข้องให้บริบทต่างกัน ได้แก่ DetectGPT (Mitchell et al., 2023) ซึ่งใช้ความโค้งของ log probability สำหรับการตรวจจับแบบ zero-shot; TuringBench (Uchendu et al., 2021) ซึ่งมี 20 ป้ายกำกับรวมมนุษย์และผู้สร้าง 19 แบบ; M4 (Wang et al., 2024) ซึ่งศึกษาหลายภาษา หลายโดเมนและหลายผู้สร้าง; และ RAID (Dugan et al., 2024) ซึ่งเน้นความทนทาน การเปรียบเทียบเหล่านี้ไม่ใช่การทดสอบโดยตรงกับข้อมูลของโครงงาน และไม่ใช้ยืนยันว่าไม่มีงานภาษาไทยอื่น

Lowphansirikul et al. (2021) เสนอ WangchanBERTa สำหรับงานประมวลผลภาษาไทยทั่วไป มิใช่แบบจำลองที่ฝึกมาเพื่อการตรวจจับข้อความปัญญาประดิษฐ์โดยเฉพาะ โครงงานจึงปรับแต่งหัวจำแนกและประเมินภายใต้ขอบเขตข้อมูลข่าวของตนเอง

ตารางที่ 2.2 การวิเคราะห์ช่องว่างทางงานวิจัย

งานวิจัย (Author)	**แนวคิดหลัก (Key Concept)**	**ข้อจำกัด (Limitation)**	**การเติมเต็มในโครงงานนี้ (Contribution)**
Mitchell et al. (2023)	การใช้ DetectGPT ตรวจจับข้อความปัญญาประดิษฐ์แบบ Zero-shot ผ่านความโค้งของฟังก์ชันความน่าจะเป็น (Probability Curvature)	มุ่งเน้นภาษาอังกฤษและต้องพึ่งพาแบบจำลองอ้างอิงขนาดใหญ่ซึ่งมีต้นทุนการคำนวณสูง	พัฒนาตัวจำแนกแบบ Supervised ภาษาไทยขนาดกะทัดรัด (INT8 ขนาด 106 MB) ที่ประมวลผลฝั่งผู้ใช้ (Client-side) บนเบราว์เซอร์ได้จริง
Lowphansirikul et al. (2021)	แบบจำลองภาษาก่อนการฝึกฝน WangchanBERTa ด้วยสถาปัตยกรรม RoBERTa/CamemBERT และ SentencePiece-style Unigram/Metaspace	เน้นงานประมวลผลภาษาธรรมชาติทั่วไป มิได้ถูกฝึกฝนหรือออกแบบมาสำหรับงานตรวจจับข้อความปัญญาประดิษฐ์	นำมาปรับแต่งหัวจำแนกแบบ 4 ประเภท (4-way Classification Head) เพื่อใช้ระบุค่ายผู้ให้บริการแบบจำลองภาษาในภาษาไทย

งานวิจัย (Author)	แนวคิดหลัก (Key Concept)	ข้อจำกัด (Limitation)	การเติมเต็มในโครงงานนี้ (Contribution)
Uchendu et al. (2021)	TuringBench — การจำแนกแหล่งกำเนิดข้อความหลายค่าย (Multi-class Attribution) ครอบคลุมแบบจำลองภาษา 19 ตัว	ศึกษาเฉพาะข้อความภาษาอังกฤษ และประเมินบนแบบจำลองภาษาขนาดใหญ่ยุคก่อน	ขยายขอบเขตการระบุค่ายผู้ให้บริการ (Vendor Attribution) สู่ภาษาไทย 4 ประเภท (มนุษย์, GPT, Gemini, Other)
Wang et al. (2024)	M4 — การตรวจจับข้อความแบบหลายแบบจำลอง หลายโดเมน และหลายภาษา (Multi-generator, Multi-domain, Multi-lingual)	ขอบเขตภาษา/โดเมน/ผู้สร้างต่างจากการทดลองข่าวไทยนี้; ไม่ใช่ direct benchmark comparison	สร้างคลังข่าวภาษาไทยสมดุล 38,428 แถว ใช้แม่แบบร่วมกันและแบ่ง gid เพื่อลดการข้ามกลุ่ม แต่ไม่ขจัดตัวแปรกวนทั้งหมด
Dugan et al. (2024)	RAID — ชุดทดสอบมาตรฐาน (Benchmark) ที่เน้นประเมินความทนทาน (Robustness) ต่อการโจมตีเชิงดัดแปลงข้อความ	ครอบคลุมเฉพาะภาษาอังกฤษ และระบบตรวจจับส่วนใหญ่ยังพึ่งพาการประมวลผลฝั่งเซิร์ฟเวอร์ (Server-side)	ประมวลผลฝั่งผู้ใช้ พร้อมตรวจ network isolation ใน staging และบรรยาย GPT magnet effect โดยไม่สรุปเหตุและผล

โครงงานมุ่งเพิ่มกรณีศึกษาการจำแนกแหล่งกำเนิดข่าวภาษาไทยแบบสี่ประเภทและการประมวลผลฝั่งผู้ใช้ โดยใช้แม่แบบคำสั่งร่วมกันและแบ่งข้อมูลระดับกลุ่มเพื่อลดความเสี่ยงบางด้าน ยังมีตัวแปรกวนด้านแหล่งข่าว ความยาว โหมดสร้างและผู้สร้างที่แยกไม่ขาด จึงไม่สรุปว่าการออกแบบทำให้เรียนรู้เฉพาะสไตล์หรือขจัดการรั่วไหลทั้งหมด

บทที่ 3
วิธีการวิจัย

ระเบียบวิธีประกอบด้วยการสร้างข้อมูลข่าวภาษาไทย การปรับแต่ง WangchanBERTa การประเมินอ้างอิง FP32 การวินิจฉัย INT8 การเลือก W8A32 ด้วย validation การแก้ parity ตัวตัดโทเคน และการประเมินแบบจำลองตรึงบน test เดิม ตามด้วยการวิเคราะห์ script-mixing เชิงสำรวจและทดสอบระบบเบราว์เซอร์ ขั้นตอนหลังไม่ใช้เลือกแบบจำลองจากคะแนน test

3.1 ภาพรวมของโครงงาน

สถาปัตยกรรมของระบบแบ่งออกเป็นสองส่วนที่แยกจากกันโดยสิ้นเชิง คือ ท่อส่งการพัฒนาแบบจำลองแบบออฟไลน์ (offline ML pipeline) ซึ่งทำงานบนเครื่องของผู้พัฒนา และ ระบบให้บริการฝั่งผู้ใช้งาน (client-side serving system) ซึ่งไม่เรียกเซิร์ฟเวอร์อนุมานข้อความของโครงงาน ส่วนบริการข้อมูลป้อนกลับและผู้ดูแลแยกต่างหาก

3.1.1 ขั้นตอนการพัฒนาแบบจำลอง (Offline Development Phase)

รวบรวมข่าวมนุษย์สองแหล่ง เป้าหมาย 10,000 บทความ สร้างฉบับ GPT/Gemini/Other ทำความสะอาดและเก็บเฉพาะกลุ่มครบสี่ประเภท แล้วแบ่งระดับบทความฝึก WangchanBERTa ด้วย GPU และ fp16 ตามตารางที่ 3.2 ส่งออก FP32 ONNX เพื่อเป็นอ้างอิง การวินิจฉัยพบ dynamic INT8 เดิมเสื่อมลง จึงทดลอง quantization ในไดเรกทอรีแยกและเลือก W8A32 ด้วย validation ไม่เขียนทับ checkpoint หรือ production R2

3.1.2 ระบบให้บริการฝั่งผู้ใช้งาน (Client-side Serving Architecture)

ระบบฉบับแก้ไขใช้ React 19/TypeScript และ Vite โดย hook จัดสถานะ ชั้นบริการโหลด manifest และตรวจขนาด/SHA256 ของแบบจำลอง ตัวตัดโทเคนและ config ใช้เส้นทาง /artifacts/thai-en-w8a32-4baa5cb331f6-v1/ บนต้นทางของเว็บเดียวกัน ไม่มีการดึง tokenizer จาก mutable Hugging Face main หรือแบบจำลอง R2 เดิม Web Worker ตรวจ runtime และ schema ก่อนสร้าง ort.InferenceSession ใช้ WASM หนึ่งเธรด ป้อน input_ids/attention_mask แบบ int64 รูป [1,400] แล้วรับ logits float32 [1,4] คำนวณ stable softmax และ argmax ค่ารวม AI คือ 1−P(Human) ไม่ใช่คะแนนประเภทที่ชนะ การดาวน์โหลดเป็นสายเดียวที่มีขอบเขตเวลาและตรวจ hash แคชเป็นทางเลือก ส่ง ArrayBuffer แบบ transferable และจัดคิวคำขอเพื่อไม่ให้สถานะซ้อนกัน

3.1.3 แบบจำลองความเป็นส่วนตัวสองโหมด (Two-Mode Privacy Model)

แยกการตรวจข้อความและข้อมูลป้อนกลับอย่างชัดเจน การอนุมานเกิดภายในเบราว์เซอร์โดยไม่ส่งข้อความให้เซิร์ฟเวอร์อนุมาน ส่วนข้อมูลป้อนกลับส่งข้อความและผล

ทำนายเมื่อผู้ใช้ยินยอม ผ่าน Turnstile และ Edge Function ที่ตรวจข้อมูลและอัตราการส่ง ผู้ดูแลต้องผ่าน Supabase Auth และ allowlist/RLS การทดสอบ inference-only Preview ที่ไม่พบคำขอ Supabase เป็นหลักฐานระยะก่อนหน้า ไม่ใช่ขอบเขตของบริการเสริมปัจจุบันหรือการรับรองการปฏิบัติตาม PDPA

3.2 การวิเคราะห์ขอบเขตและความต้องการของระบบ

3.2.1 ความต้องการด้านฟังก์ชันการทำงาน (Functional Requirements)

- ระบบต้องจำแนกข้อความภาษาไทยออกเป็น 4 ประเภท (Human / GPT / Gemini / Other) พร้อมแสดงการแจกแจงความน่าจะเป็นครบทุกประเภท
- ระบบรับข้อความ 200–10,000 ตัวอักษรเป็นข้อกำหนด UI ไม่ใช่หลักประกันว่าข้อความทุกความยาวมีบริบทพอ แบบจำลองใช้เพียงไม่เกิน 398 content tokens ก่อน BOS/EOS รวมเป็น 400 โทเคน จึงอาจไม่เห็นข้อความส่วนท้าย
- ระบบต้องแสดงคำตัดสินหลักหนึ่งค่าที่ชัดเจน (ประเภทที่ชนะพร้อมระดับความเชื่อมั่น) ก่อนรายละเอียดเชิงลึก ตามคำแนะนำของอาจารย์ที่ปรึกษา
- ระบบรองรับข้อมูลป้อนกลับแบบยินยอมผ่านการตรวจ CAPTCHA และข้อมูลฝั่งบริการ ส่วนผู้ดูแลต้องผ่าน Supabase Auth และการอนุญาตในฐานข้อมูล ไม่ใช้รหัสผ่านที่ฝังในชุดไฟล์เว็บ
- ส่วนติดต่อผู้ใช้ต้องเป็นสองภาษา (ไทยเป็นหลัก อังกฤษเป็นรอง) และรองรับโหมดสว่าง/มืด

3.2.2 ความต้องการด้านคุณภาพและความปลอดภัย (Non-functional Requirements)

- ความเป็นส่วนตัว: ออกแบบไม่ให้ส่งข้อความผู้ใช้ระหว่าง inference และตรวจคำขอเครือข่ายในกรณีทดสอบ การส่งข้อมูลป้อนกลับเป็นการกระทำสมัครใจที่ขอความยินยอมแยกต่างหาก และการขัดข้องของ backend ต้องไม่กระทบการอนุมาน
- ต้นทุน: ลดการใช้เซิร์ฟเวอร์อนุมาน ใช้บริการภายใต้โควตาที่มีอยู่ แต่ต้องคำนึงถึงค่าเก็บไฟล์ CDN แบนด์วิดท์และเครื่องผู้ใช้ ไม่รับประกันต้นทุนศูนย์ในทุกขนาดการใช้งาน

- ประสิทธิภาพ: กำหนดเป้าหมาย warm inference บนเครื่องสาธิตพร้อมสื่อสารสถานะ การโหลดและเริ่ม Worker ต้องมีเวลาจำกัด ตรวจ integrity และให้ผู้ใช้ลองใหม่ได้เมื่อผิดพลาด ผลวัดตามสภาพทดสอบแสดงในบทที่ 4
- ความเข้ากันได้: ตรวจจริงบน desktop Edge 157.0.4341.42, Chrome 157.0.8103.39 และ Firefox 159.0.1 โดยใช้ pinned ONNX Runtime Web การรองรับ WASM/SIMD เป็นเงื่อนไขเทคนิค แต่ไม่ทดแทนการทดสอบรุ่นเก่า Safari มือถือ offline และเครื่องหน่วยความจำน้อย
- ความโปร่งใส (Transparency): ผลวิเคราะห์ต้องแสดงการแจกแจง softmax ครบ 4 ประเภทเสมอ และนำเสนอการระบุค่ายเป็น "ค่ายที่น่าจะเป็น" มิใช่ข้อสรุปเด็ดขาด

3.2.3 ความต้องการด้านสภาพแวดล้อมทางเทคนิค (Technical Requirements)

- เว็บแอป: React 19, TypeScript, Vite และ Tailwind CSS v4; onnxruntime-web ตรึงรุ่น 1.26.0-dev.20260416-b7804b056c; @huggingface/tokenizers 0.1.3 พร้อม public normalizer extension สำหรับ charsmap เดิม ไม่มีการพึ่ง _tokenizer ภายในแบบถาวร ส่วน Supabase client ใช้เฉพาะระบบผู้ดูแลและข้อมูลป้อนกลับ
- ฝั่งท่อส่งข้อมูลและแบบจำลอง: Python 3.11, Hugging Face Transformers/Datasets/Accelerate, scikit-learn, SentencePiece, Optimum + ONNX Runtime (ส่งออกและบีบอัด), API ของ OpenAI / Google GenAI / OpenRouter (สร้างข้อมูลสังเคราะห์)
- เผยแพร่ไฟล์ static รวม model/tokenizer/runtime บนต้นทางเดียวกันด้วย Vercel โดยตรวจ SHA256 ก่อนใช้งาน รุ่นที่รวม security ผ่านการตรวจแบบ staged ก่อนเผยแพร่ และแยกการปรับ source จากการ promote ด้วยเวิร์กโฟลว์ manual release ส่วน R2 เดิมไม่ถูกเขียนทับ GitHub Actions keep-alive เดิมเลิกอ่านข้อมูลป้อนกลับแบบ anonymous แล้ว และเก็บตัวอย่างเก่าไว้ในภาคผนวกเพื่อประวัติเท่านั้น
- เครื่องฝึกฝนแบบจำลอง: Acer Predator Helios Neo 16 Intel Core i7-13700HX, RAM 32 GB, NVIDIA GeForce RTX 4050 Laptop (VRAM 6 GB)

3.3 การดำเนินงาน

พัฒนาการเดิมประกอบด้วย V1 ระยะ SP1 บนข้อมูลนำร่อง 670 ตัวอย่าง และ V2 ที่ขยายการจำแนก ก่อนสร้าง V3 สมดุลสี่ประเภทในรายงานนี้ ตัวเลขประวัติ V1/V2 ไม่ใช่ผลเปรียบเทียบโดยตรงบน test เดียวกัน ผลหลักฉบับสมบูรณ์ยึด W8A32 ที่เลือกด้วย validation และประเมิน test เดิมภายหลัง

3.3.1 แหล่งข้อมูลมนุษย์และการลดอคติเชิงหัวข้อ (Topic De-biasing)

ข้อความมนุษย์รวบรวมจากสองแหล่ง รวม 10,000 บทความ 1. The Standard จำนวน 5,000 บทความ (หมวดเทคโนโลยีและต่างประเทศ) รวบรวมด้วยการไต่หน้ารายการบทความ (listing-page crawl) ด้วยไลบรารี requests และ BeautifulSoup 2. Sanook จำนวน 5,000 บทความ (ข่าวทั่วไปหลากหลายหมวด) รวบรวมผ่าน ดัชนีแผนผังเว็บไซต์แบบ XML (XML sitemap index เฉพาะชุด news_original_*.xml.gz ซึ่งเป็นงานข่าวที่กองบรรณาธิการ Sanook ผลิตเอง ไม่รวมเนื้อหารับช่วงจากสำนักอื่น) วิธีนี้หลีกเลี่ยงระบบแบ่งหน้าแบบ JavaScript ของเว็บไซต์ได้โดยไม่ต้องใช้เครื่องมือจำลองเบราว์เซอร์

การใช้ Sanook และ The Standard ช่วยเพิ่มความหลากหลายของแหล่งข่าวและทำเนียบภาษาเมื่อเทียบกับแหล่งเดียว แต่ไม่ได้ทำให้หัวข้อและแหล่งข้อมูลเป็นอิสระจากป้ายกำกับโดยสมบูรณ์ ไม่มีหมวดหัวข้อที่ผ่านการตรวจสอบใน CSV สุดท้าย จึงใช้ source เป็นตัวแปรประกอบการวิเคราะห์ ไม่อ้างว่าสามารถบังคับให้แบบจำลองเรียนรู้เฉพาะสไตล์

3.3.2 การขยายบทความและการควบคุมคำสั่งสร้างข้อความ (Per-article Expansion & Prompt Control)

บทความตั้งต้นแต่ละชิ้นถูกกำกับด้วยรหัสกลุ่ม (`gid` รูปแบบ `sanook_<id>` / `ts_<id>`) และขยายเป็น 4 แถว หนึ่งแถวต่อประเภท ดังตารางที่ 3.1

ตารางที่ 3.1 โครงสร้างประเภทและแบบจำลองสร้างข้อความ

ประเภท	ชื่อ	ผู้สร้างข้อความ	จำนวนที่สร้าง (บทความ)
0	Human	นักข่าว Sanook + The Standard	10,000
1	GPT	`gpt-4o-mini` (OpenAI API แบบเรียลไทม์)	10,000
2	Gemini	`gemini-2.5-flash-lite` (Google GenAI API)	10,000
3	Other	`deepseek/deepseek-chat` (DeepSeek-V3) 5,000 + `qwen/qwen-2.5-7b-instruct` 5,000 ผ่าน OpenRouter	10,000

แต่ละบทความได้รับโหมด polish หรือ pure จากการจัดสรรในต้นทาง โดย polish ใช้เนื้อข่าวมนุษย์ให้เรียบเรียงใหม่ ส่วน pure ใช้พาดหัวให้สร้างข่าว การคงเฉพาะกลุ่มครบสี่ประเภทและการใช้โควตา Other ทำให้การแจกแจงสุดท้ายมิใช่การทดลองแฟกทอเรียลที่ควบคุมผู้สร้างกับโหมดแยกจากกัน ใน test พบ DeepSeek 495 ตัวอย่าง (polish 488/pure 7) และ Qwen 466 ตัวอย่าง (polish 7/pure 459) จึงต้องอภิปรายความสับสนระหว่างผู้สร้างและโหมด ประวัติการลอง Typhoon เป็นการพัฒนาก่อนหน้า มิได้รวมเป็นประเภท Other สุดท้าย

แม่แบบคำสั่ง polish/pure ใช้ร่วมกันจาก scripts/gen_prompts.py แต่ไม่ได้ยืนยันว่าคำขอ API และ sampling ทุกค่ายเหมือนกัน OpenAI/OpenRouter ระบุ temperature 0.8 และ max_tokens 900 ขณะที่ generate_gemini.py เรียก generate_content โดยไม่ส่ง generation config แม้ import ค่าคงที่ร่วมกัน ค่าเริ่มต้นของบริการและความแตกต่างของผู้ให้บริการจึงยังเป็นตัวแปรกวน

3.3.3 การทำความสะอาดข้อมูล (Data Cleaning Filters)

ตัวกรองห้าขั้นต่อไปนี้ถูกประยุกต์อย่างอิสระต่อแต่ละประเภทข้อมูล เพื่อขจัด "สัญญาณบอกใบ้" (tells) เชิงรูปแบบที่มิใช่สไตล์การประพันธ์ที่แท้จริง

1. ลบโครงสร้าง Markdown: เครื่องหมายเน้นข้อความ (`**`, `*`), อักขระ backtick และหัวข้อ (`#`) ซึ่งหากคงไว้จะกลายเป็นลายนิ้วมือปลอมของฝั่งปัญญาประดิษฐ์

2. ลบป้ายโครงนั่งร้านของแบบจำลองขนาดเล็ก: เช่น "คำนำ:", "เนื้อข่าว:", "Here is..." ซึ่งเป็นรูปแบบสำเร็จที่หลุดมากับผลลัพธ์ของแบบจำลองเปิดเผยค่าน้ำหนัก

3. ตัวกรองความยาว: ขั้นต่ำ 200 ตัวอักษร และตัดทอนที่ 3,000 ตัวอักษร

4. ตัวกรองสัดส่วนอักขระไทย $\geq$ 0.5: ทิ้งผลลัพธ์เสื่อมสภาพที่เป็นภาษาอังกฤษหรือภาษาผสมมากผิดปกติ เพื่อมิให้ตัวจำแนกเรียนรู้ "สัญญาณภาษา" แทนสไตล์การประพันธ์

5. ขจัดข้อความซ้ำแบบตรงตัว

3.3.4 ความสมดุลโดยการสร้าง (Balance by Construction)

เงื่อนไขการคงข้อมูลกำหนดว่า บทความ (`gid`) จะถูกเก็บไว้ก็ต่อเมื่อข้อความรอดตัวกรองครบทั้ง 4 ประเภท บทความที่ผ่านเกณฑ์แต่ละชิ้นจึงมีส่วนร่วม 1 แถวต่อประเภทพอดี ทำให้ชุดข้อมูล สมดุล 4 ทางโดยการสร้าง (perfectly balanced by construction) โดยไม่ต้องสุ่มลดหรือเพิ่มตัวอย่างภายหลัง จากบทความเป้าหมาย 10,000 ชิ้น มีบทความที่รอดตัวกรองครบทุกประเภท 9,607 ชิ้น ได้ชุดข้อมูลสุดท้าย 38,428 แถว (รายละเอียดผลลัพธ์ในหัวข้อ 4.1)

3.3.5 การป้องกันข้อมูลรั่วไหลด้วยการแบ่งข้อมูลระดับกลุ่ม (GroupShuffleSplit)

โครงสร้างชุดข้อมูลของโครงงานมีคุณสมบัติพิเศษที่เป็นทั้งจุดแข็งและความเสี่ยงคือ ทุกบทความตั้งต้นแตกตัวเป็น 4 แถวที่ผูกพันเชิงเนื้อหาอย่างแนบแน่น (ต้นฉบับมนุษย์ + ฉบับเรียบเรียง/เขียนใหม่โดยปัญญาประดิษฐ์ 3 ค่ายจากบทความเดียวกัน) หากแบ่งชุดข้อมูลด้วยการสุ่มระดับแถวตามวิธีทั่วไป บทความหนึ่ง ๆ ย่อมมีบางแถวตกอยู่ในชุดฝึกสอนและบางแถวตกอยู่ในชุดทดสอบ แบบจำลองจะสามารถอาศัยการจดจำ เนื้อหา ของบทความ (ชื่อบุคคล เหตุการณ์ ตัวเลขในข่าว) ที่เคยเห็นในชุดฝึกสอน มาช่วยจำแนกแถวพี่น้องในชุดทดสอบ ทำให้ตัวเลขประเมินที่ได้จะสูงเกินความสามารถจริงในการจำแนก สไตล์การประพันธ์ เป็นการรั่วไหลของข้อมูล (data leakage) ที่ตรวจพบยากเพราะไม่มีแถวใดซ้ำกันตรง ๆ เลย

โครงงานป้องกันการรั่วไหลดังกล่าว ชั้นควบคุมตัวแปรกวนชั้นที่สาม ด้วยการแบ่งข้อมูลเป็น ระดับบทความ ไม่ใช่ระดับแถว ตามหลักการเดียวกับ `GroupShuffleSplit` ของไลบรารี scikit-learn โดยมีขั้นตอนดังนี้

1. รวบรวมรหัสบทความ (gid) ที่ไม่ซ้ำทั้งหมด 9,607 ค่า

2. สับลำดับรายการ gid ด้วยตัวกำเนิดเลขสุ่มที่ตรึงค่าเริ่มต้น (random_state = 42) เพื่อให้ผลการแบ่งทำซ้ำได้ (reproducible)

3. แบ่งรายการ gid ออกเป็นสามส่วนตามสัดส่วน 80/10/10 จะได้ ชุดฝึกสอน 7,685 บทความ, ชุดทวนสอบ 961 บทความ, ชุดทดสอบ 961 บทความ

4. จัดสรรแถวข้อมูลทั้ง 4 แถวของแต่ละบทความเข้าสู่ส่วนเดียวกับ gid ของบทความนั้นเสมอ

5. การยืนยันภายหลัง (post-hoc assertion): สคริปต์ประกอบชุดข้อมูลตรวจสอบโดยอัตโนมัติว่าไม่มี gid ใดปรากฏในมากกว่าหนึ่งส่วนข้อมูล หากพบจะยุติการทำงานทันที

การแบ่งตาม gid ทำให้ต้นฉบับและฉบับ AI จากบทความรหัสเดียวกันอยู่ใน split เดียวกัน การกู้คืน lineage ของข้อมูลปัจจุบันพบ gid ไม่ทับซ้อนและครบสี่ประเภททุกกลุ่ม แต่ไม่ได้พิสูจน์ว่าไม่มีข่าวเรื่องเดียวกันคนละรหัส ข้อความใกล้เคียงเชิงความหมาย หรือข้อมูลที่มีอยู่ใน pretraining ผลจึงเป็นการประเมินภายใต้การแบ่งระดับบทความ มิใช่ชุดที่รับรองว่าปลอดการปนเปื้อนทุกชนิด

3.3.6 การฝึกฝนแบบจำลอง (Model Fine-tuning)

การฝึกเดิมใช้ WangchanBERTa และ fp16 บน RTX 4050 Laptop ตามพารามิเตอร์ตารางที่ 3.2 มี checkpoint ที่ดีที่สุด checkpoint-5766 ณ epoch 3 ซึ่งน้ำหนักตรงกับ model_output/model.safetensors ส่วน epoch สุดท้ายมิใช่รุ่นที่เลือก การวินิจฉัยภายหลังอ่าน checkpoint และประเมินโดยไม่ฝึกซ้ำ

ตารางที่ 3.2 อภิมานพารามิเตอร์การฝึกสอนแบบจำลอง V3

อภิมานพารามิเตอร์	ค่า	หมายเหตุ
อัตราการเรียนรู้ (learning rate)	1×10^{-5}	ลดจาก 2×10^{-5} ของรุ่น V2 เพื่อบรรเทาการเรียนรู้เกินพอดี (overfitting)
สัดส่วนการอุ่นเครื่อง (warmup ratio)	0.1	อุ่นอัตราการเรียนรู้ช่วงร้อยละ 10 แรก เพื่อเสถียรภาพช่วงต้นการฝึก
ขนาดชุดย่อย (batch size)	8 × การสะสมเกรเดียนต์ 2 ขั้น = ประสิทธิผล 16	กำหนดตามขีดจำกัด VRAM 6 GB
จำนวนรอบ (epochs)	4 พร้อม Early Stopping (patience = 2 บน Macro F1)	โหลดแบบจำลองที่ดีที่สุดเมื่อสิ้นสุด (`load_best_model_at_end`)
การลดทอนน้ำหนัก (weight decay)	0.01	-
ความยาวลำดับสูงสุด (max length)	400 โทเคน (ฝึก/ประเมิน)	ฝึก ประเมิน และเบราว์เซอร์ใช้ 400 โทเคนรวม BOS/EOS; ไม่ใช้ 416
ความละเอียดเชิงตัวเลข	fp16 mixed precision	-

3.3.7 การส่งออก บีบอัด และนำขึ้นใช้งาน (Export, Quantization & Deployment)

การส่งออกเดิมใช้ Optimum ได้ FP32 ONNX ประมาณ 421.21 MB ต่อมา dynamic INT8 แบบ per-tensor ขนาด 106,066,136 ไบต์ให้ผลจำแนกเสื่อมลงบนอินพุตเดียวกัน การทดลองแก้ไขจึงสร้างไฟล์แยกโดยคงต้นฉบับ เลือก wo_pc_foldable.onnx ขนาด 106,299,948 ไบต์จากผล validation และความเข้ากันได้ WASM แล้วจัดชุดแบบ immutable สำหรับ Preview และรุ่นเผยแพร่ โดยไม่เขียนทับ R2 เดิม รายละเอียดการวินิจฉัยและเลือกแบบจำลองอยู่ในหัวข้อถัดไป

3.3.8 การวินิจฉัยความคลาดเคลื่อนและเลือก W8A32

ใช้ frozen Python token IDs/masks เดียวกันสำหรับเปรียบเทียบ PyTorch FP32 และ FP32 ONNX ซึ่งให้ class prediction ตรงกันทั้ง 3,844 test ตัวอย่างและ Macro F1 ร้อยละ 87.5074 ส่วน dynamic INT8 เดิมได้ร้อยละ 75.8756 และเปลี่ยน prediction 529 ตัวอย่าง พบ reconstruction error ของ per-tensor weights มากใน FFN output โดยเฉพาะชั้น 6–10 แต่การแก้ per-channel เพียงอย่างเดียวยังไม่พอ คง activation-only

attention, embeddings และ FFN output เป็น FP32 ทำให้ผลดีขึ้นตามชุดทดลอง จึงไม่อ้างว่าจุดเดียวอธิบายทุก numerical divergence

ขั้นเลือก configuration ใช้ data/val.csv 3,844 ตัวอย่างเท่านั้น ทดลอง broad/constant-only dynamic per-channel, เลือกยกเว้น embeddings/FFN และ weight-only FP32-activation รวมทั้ง Cast/Mul foldable เปรียบเทียบกับ FP32 baseline รายละเอียดผล validation แสดงตารางที่ 4.5 เลือก W8A32 เพราะขนาด 106.30 MB และ numerical agreement กับ WASM ดีกว่า mixed dynamic ที่ใหญ่กว่า มิใช่เพราะ final-test score สูงสุด ไม่มีการ retrain หรือปรับ threshold จาก test

3.3.9 ความเท่าเทียมของตัวตัดโทเคนและการประเมินตรึงขั้นสุดท้าย

สัญญาอินพุตกำหนด 398 content tokens ตามด้วยการประกอบ BOS=5/EOS=6 รวมไม่เกิน 400 แล้ว right-pad ด้วย PAD=1 และ mask 1/0 การแก้จาก 416 เป็น 400 อย่างเดียวไม่พอ เพราะอาจสูญ EOS และ normalizer JavaScript เดิมใช้ NFKC แทน saved charsmap การแก้ถาวรใช้ public normalizer extension ของ @huggingface/tokenizers 0.1.3 พร้อมตรึงไฟล์และทดสอบ ID/mask/hash เทียบ Python ครบ validation 3,844 แถวและ edge cases 15 กรณี (ไม่ตรงกัน 0 กรณี) ไม่ใช่หลักประกัน Unicode ทุกแบบ

ตรึง artifact SHA256 4baa5cb331f6cad7d90492d33c3e9a5bf53a45029ca5a563aaa86f7a46bc84a2 และ class order [Human,GPT,Gemini,Other] ก่อนประเมิน test เดิมหนึ่งรอบ 3,844 calls ใช้ raw CSV ไม่ trim เพิ่ม Python fast tokenizer และ [1,400] int64 CPUExecutionProvider/ONNX Runtime 1.24.3, batch 1, intra-op 4, argmax logits ไม่ปรับ decision threshold ใช้ scikit-learn 1.7.2 แบบ macro เฉลี่ยรายคลาสและ zero_division=0 ตรวจ confusion matrix ซ้ำจาก counts ได้ตรงกัน การตรวจ input parity ของ JS บน test เดิมไม่ใช่การคัดเลือก configuration เพิ่ม

บทที่ 4
ผลการดำเนินงาน

4.1 ผลการจัดเตรียมข้อมูล

จากบทความตั้งต้นเป้าหมาย 10,000 บทความ (Sanook 5,000 + The Standard 5,000) กระบวนการสร้างข้อความสังเคราะห์สามค่ายและการกรองคุณภาพห้าขั้น (หัวข้อ 3.3.3) ให้ผลลัพธ์ดังนี้: บทความที่ข้อความรอดตัวกรอง**ครบทั้งสี่ประเภท**มีจำนวน 9,607 บทความ (ร้อยละ 96.07 ของเป้าหมาย) แต่ละบทความมีส่วนร่วมหนึ่งแถวต่อประเภทพอดี ได้ชุดข้อมูลสุดท้าย 38,428 แถว สมดุลสมบูรณ์ทั้งสี่ประเภท (9,607 แถวต่อประเภท) โดยไม่ต้องสุ่มลดหรือสุ่มเพิ่มตัวอย่าง การแบ่งชุดข้อมูลระดับกลุ่มบทความตามหัวข้อ 3.3.5 ให้ผลการแบ่งดังตารางที่ 4.1 (ตัวเลขตรวจนับจากไฟล์ `data/{train,val,test}.csv` จริง)

ตารางที่ 4.1 สรุปชุดข้อมูล V3 หลังการแบ่งระดับกลุ่มบทความ

ส่วนข้อมูล	**จำนวนแถว (แถว)**	**ต่อประเภท (**0/1/2/3)	**จำนวนบทความ (บทความ)**
ชุดฝึกสอน (train)	30,740	7,685 เท่ากันทุกประเภท	7,685
ชุดทวนสอบ (validation)	3,844	961 เท่ากันทุกประเภท	961
ชุดทดสอบ (test)	3,844	961 เท่ากันทุกประเภท	961
รวม	38,428	9,607 ต่อประเภท	9,607

คุณสมบัติเชิงระเบียบวิธีของชุดข้อมูลที่ได้ สรุปได้สามข้อ

1. ทุก split มีจำนวนตัวอย่างต่อประเภทเท่ากัน ทำให้ macro recall เท่ากับ accuracy ในข้อมูลนี้ แต่ Macro F1 ไม่จำเป็นต้องเท่ากับ Micro F1 และไม่ใช่ harmonic mean ของ Macro Precision กับ Macro Recall

2. การกู้คืนรหัสต้นทางยืนยันการไม่ข้าม split ของ gid ในข้อมูลปัจจุบัน ไม่ใช่การตรวจ semantic near-duplicate
หรือการรับรองว่าบทความไม่มีความเกี่ยวข้องกันข้ามรหัส

3. แหล่งข่าวสองแห่ง แม่แบบคำสั่งกลาง
และตัวกรองอักขระไทยเป็นมาตรการลดความแตกต่างบางด้าน แต่ยังพบตัวแปรกวนด้าน source ความยาว โหมด และผู้สร้าง โดยเฉพาะ Other-generator กับ polish/pure
จึงไม่สรุปว่าความแม่นยำทั้งหมดมาจากการเรียนรู้สไตล์

4.2 ผลการทดสอบประสิทธิภาพแบบจำลอง

แยกผลอ้างอิง FP32 ผล validation สำหรับเลือก quantization และผล test ของ W8A32 ที่ตรึงแล้วออกจากกัน ตาราง 4.2 ใช้ผล controlled evaluation ที่เก็บไว้ ไม่แทนด้วยค่า Trainer validation ดั้งเดิม test เดิม 3,844 ตัวอย่างเคยถูกตรวจระหว่างวินิจฉัยรุ่นก่อน จึงไม่ใช่ blind test ที่ไม่เคยเปิดดู การประเมิน W8A32 สุดท้ายทำหนึ่งรอบโดยไม่เลือก configuration จากผล test หลักฐานการประเมินและการทำซ้ำอยู่ในภาคผนวก ค

4.2.1 ตัวชี้วัดรวม (Aggregate Metrics)

ตารางที่ 4.2 ตัวชี้วัดรวมบนชุดทวนสอบและชุดทดสอบ

แบบจำลอง / split (N=3,844)	Accuracy %	Macro P %	Macro R %	Macro F1 %
FP32 / test อ้างอิง	87.4870	89.0007	87.4870	87.5074
FP32 / validation ควบคุม	88.7357	90.0948	88.7357	88.7861
W8A32 / validation เลือก configuration	88.7877	90.1381	88.7877	88.8342
W8A32 / test สุดท้าย	87.3569	88.8430	87.3569	87.3705

W8A32 ได้ test Macro F1 ร้อยละ 87.3705 สูงกว่าเป้าหมายร้อยละ 85 อยู่ 2.3705 จุดร้อยละ และ Human Precision ร้อยละ 97.3624 สูงกว่าเป้าหมายร้อยละ 95 เมื่อเทียบ FP32 test ค่า Macro F1 ลด 0.1369 จุดร้อยละ มี prediction เปลี่ยน 20 จาก 3,844 ตัวอย่าง (ตรงกันร้อยละ 99.4797): ถูกเป็นผิด 12 ผิดเป็นถูก 7 และผิดทั้งคู่แต่ต่างประเภท 1 ผลต่างขนาดเล็กนี้ไม่ใช่หลักฐานนัยสำคัญทางสถิติหรือการรับรอง generalization และไม่ใช้เลือกแบบจำลองใหม่

4.2.2 เมทริกซ์ความสับสน (Confusion Matrix)

ตารางที่ 4.3 เมทริกซ์ความสับสนบนชุดทดสอบ (row = ประเภทจริง, column = ประเภทที่ทำนาย; n = 961 ต่อประเภท)

จริง / ทำนาย	Human	GPT	Gemini	Other
Human	849	66	38	8
GPT	9	919	22	11
Gemini	9	56	884	12
Other	5	170	80	706

4.2.3 ตัวชี้วัดรายประเภท (Per-class Metrics)

ตารางที่ 4.4 ตัวชี้วัดรายประเภทบนชุดทดสอบ

ประเภท (support 961)	Precision %	Recall %	F1 %
Human	97.3624	88.3455	92.6350
GPT	75.8877	95.6296	84.6225
Gemini	86.3281	91.9875	89.0680
Other	95.7938	73.4651	83.1567

Human Precision = 849/(849+9+9+5) = ร้อยละ 97.3624 หมายถึงสัดส่วนมนุษย์จริงในกลุ่ม 872 ข้อความที่ทำนายเป็น Human ไม่ใช่อัตราป้องกันการกล่าวหามนุษย์ผิด ข้อความ AI ที่ถูกทำนายเป็น Human มี 23 จาก AI จริง 2,883 ตัวอย่าง (ร้อยละ 0.7978) ซึ่งเป็นอีกตัวหารหนึ่ง

Human Recall = 849/961 = ร้อยละ 88.3455 ส่วนอัตรากล่าวหามนุษย์ผิดเมื่อถือว่าการทำนายเป็น AI ประเภทใดก็ตามคือการกล่าวหา เท่ากับ (66+38+8)/961 = 112/961 = ร้อยละ 11.6545 หรือ 1—Human Recall ค่านี้ขึ้นกับประชากรข่าวที่ประเมิน ไม่ควรตีความ softmax เป็นความแน่นอนหรือใช้ผลตัดสินผู้เขียนโดยลำพัง

4.2.4 การเกิดปรากฏการณ์แม่เหล็ก GPT (The GPT Magnet Effect)

ใน W8A32 test พบ Other ⟶ GPT 170 ตัวอย่าง (ร้อยละ 17.6899 ของ Other), Human ⟶ GPT 66 และ Gemini ⟶ GPT 56 ขณะที่ GPT ⟶ Other 11 ตัวอย่าง GPT มี Precision ร้อยละ 75.8877 และ Recall ร้อยละ 95.6296 จึงใช้ชื่อเชิงพรรณนา GPT magnet effect สำหรับรูปแบบการทำนายเข้าสู่ GPT มาก ไม่ได้หมายถึงกลไกเชิงสาเหตุที่พิสูจน์แล้ว

สมมติฐานเรื่องรูปแบบข่าวที่ทับซ้อนกัน distillation หรือ alignment ยังไม่ได้รับการทดสอบเชิงเหตุและผล เมทริกซ์ความสับสนไม่สามารถระบุข้อมูลฝึกของผู้สร้างอื่นหรือพิสูจน์ว่าเป็น subset ใน style space ได้ อีกทั้ง generator/mode, source และ length มีความสัมพันธ์กัน จึงต้องแยกข้อสังเกตจากคำอธิบายที่ยังเป็นสมมติฐาน

Other มี Precision ร้อยละ 95.7938 แต่ Recall ร้อยละ 73.4651 โดย 170 ตัวอย่างไป GPT และ 80 ไป Gemini การมี Precision สูงกับ Recall ต่ำเป็นลักษณะที่สังเกตในชุดนี้ มิได้พิสูจน์ว่าผู้สร้างเลียนแบบ GPT หรือว่าตัวอย่างที่ถูกทำนายเป็น Other ถูกต้องในทุกประชากร

4.2.5 นัยต่อการออกแบบส่วนแสดงผล

UI แสดง argmax verdict พร้อมคะแนนประเภทที่ชนะและ probabilities ครบ 4 ประเภท ค่า 1—P(Human) เป็นผลรวม AI softmax ตามนิยาม ไม่ใช่ค่าที่ได้รับการสอบเทียบความน่าจะเป็นแล้ว ข้อสรุปเรื่องการตรวจจับแบบสองประเภทต้องประเมินด้วยตัวชี้วัดของโจทย์นั้นโดยชัดแจ้ง ไม่ใช้ความสับสนระหว่างค่ายยืนยันว่าการกล่าวหามนุษย์ผิดมีน้อย ระบบควรเป็นเครื่องมือประกอบการพิจารณา

4.3 ผลการวินิจฉัยและเปรียบเทียบ quantization

ตารางที่ 4.5 การเลือก configuration ด้วย validation 3,844 ตัวอย่าง (MB ฐานสิบ)

configuration (validation)	Macro F1 %	ขนาด MB	WASM ที่บันทึก
FP32 baseline	88.7861	421.21	64 inputs ผ่าน
Dynamic per-channel broad	73.7109	106.57	1-input smoke
Dynamic constant weights only	77.6617	106.53	ไม่ทดสอบ
Dynamic + FP32 embeddings/FFN 6–10	84.8733	200.68	ไม่ทดสอบ
Dynamic + FP32 embeddings/FFN ทั้งหมด	89.0506	250.18	64 inputs ผ่าน
W8A32 foldable (เลือก)	88.8342	106.30	64 inputs ผ่าน

ผลทุกแถวเป็น full validation แยกจาก screen 64 ตัวอย่างและ browser smoke การที่ mixed dynamic ได้ร้อยละ 89.0506 ไม่ทำให้เป็นแบบจำลองสุดท้ายโดยอัตโนมัติ ต้องพิจารณาขนาดและ logit drift ข้าม runtime ด้วย W8A32 เปลี่ยน validation predictions จาก FP32 15 ตัวอย่าง ส่วน final test เปลี่ยน 20 ตัวอย่าง เป็นคนละชุดและไม่ใช้ test เลือกใหม่

4.4 ผลสำรวจการปะปนอักษรไทย–ละติน

ใช้ saved final W8A32 predictions เดิมโดยไม่มี inference เพิ่ม นิยาม T เป็นอักษร/เครื่องหมายประกอบอักษรไทย และ L เป็นอักษร ASCII Latin หลังตัด URL/email; Latin share = L/(T+L) ต่างจากตัวกรอง Thai-block/all characters ของ builder การนับ Latin chunk และการเปลี่ยนสคริปต์เป็น proxy ไม่ใช่คำอังกฤษหรือช่วงสลับภาษาที่ผู้เชี่ยวชาญกำกับชื่อเฉพาะ คำยืมและอักษรโรมันของภาษาอื่นทำให้คลาดเคลื่อนได้

ตารางที่ 4.6 คุณลักษณะข้อมูลแต่ละ split (ค่าเฉลี่ยรายเอกสาร)

split	N	มี Latin %	Thai/อักขระทั้งหมดเฉลี่ย %	Latin share เฉลี่ย %	script transitions เฉลี่ย
train	30,740	66.19	91.62	2.81	9.32
validation	3,844	65.97	91.84	2.66	9.00
test	3,844	66.36	91.41	3.01	9.32

ตารางที่ 4.7 ผลกลุ่ม script-mixing ของ test เดิมตามเอกสารเต็ม

Latin share กลุ่มเอกสารเต็ม	N test	Human/GPT/Gemini/Other	Macro F1 %	Human Recall %
0	1,293	172/412/384/325	88.64	84.88
>0–<1%	512	138/121/139/114	89.21	92.03
1–<5%	1,163	379/254/248/282	86.33	87.86
5–<15%	787	245/155/178/209	84.92	90.20
15–<50%	89	27/19/12/31	77.93	81.48
≥50%	0	0/0/0/0	N/A	N/A

กลุ่ม 15–<50% มีเพียง 89 ตัวอย่าง (Gemini 12) และไม่มี Latin-majority เลย จึงไม่สรุปแนวโน้มเชิงเหตุและผล เมื่อใช้เฉพาะ source prefix ที่อยู่ใน 400-token window มี 103 ตัวอย่างเปลี่ยนกลุ่ม; moderate ได้ Macro F1 84.75% และ substantial 81.57% (N=96) จำนวน 864 แถวมี attended tokens เต็ม 400 และ 856 แถวมีอักษรนอก prefix ที่โมเดลเห็น

ตารางที่ 4.8 ตัวแปรกวนและผลแยกตามโหมดของบทความต้นทาง

กลุ่ม	N	Macro F1 %	Human Recall %
polish-associated	1,980	83.33	88.48
pure-associated	1,864	91.63	88.20
Sanook	1,848	87.20	90.04
The Standard	1,996	87.52	86.77

โหมดของแถว Human เป็นโหมดที่กำหนดให้บทความเพื่อสร้างฉบับ AI ไม่ได้หมายความว่ามนุษย์ถูก AI polish แหล่งข่าว ความยาวและสัดส่วนคลาสต่างกันระหว่างกลุ่ม และ Other generator เกือบแยกตามโหมด (DeepSeek polish 488/495; Qwen pure 459/466) ไม่มี topic label ที่ยืนยัน การเปรียบเทียบไม่พิสูจน์ผลของ code-switching โดยตรง ข้อเสนอ pilot 480 ข้อความเป็นงานอนาคต ยังไม่เก็บหรือประเมิน

4.5 ผลทดสอบเว็บแอปพลิเคชันและ HTTPS Preview

ผลการทดสอบ inference-only ระยะก่อนหน้า: Regression เดิมผ่าน 16 tests รวม tokenizer parity validation 3,844 + edge cases 15 TypeScript/ESLint/build ผ่าน และ clean npm install/build ใน workspace แยกด้วย Node 24.22.1/npm 11.19.1 ทำซ้ำ staging-build-v4 ได้ 16 ไฟล์ตรง hash ทุกรายการ ตาราง 4.9 เป็น local complete-app smoke คนละการทดลองกับ isolated worker readiness และ HTTPS จึงไม่รวมเวลาเป็นผลเดียว

ตารางที่ 4.9 Local complete-app smoke: warm n=4 ต่อเบราว์เซอร์

browser (local)	fresh ready s	cached ready s	warm median s	warm p95 s
Edge 157.0.4341.42	2.086	1.090	1.671	1.698
Chrome 157.0.8103.39	1.777	1.165	1.692	1.868
Firefox 159.0.1	1.957	1.318	1.841	1.852

ที่มา: จัดทำโดยผู้จัดทำจากผลการทดสอบเว็บแอปพลิเคชันบนเครื่องทดสอบ

ผล Edge เป็นข้อมูลฐานที่เก็บไว้ก่อน Chrome และ Firefox ส่วน Firefox ในการทดสอบบนเครื่องมี preload diagnostic หนึ่งรายการที่แยกวินิจฉัย จึงมิใช่การรับรองว่า log ว่างทุกรอบการทดสอบ

ตารางที่ 4.10 Protected HTTPS Preview: warm n=4 ต่อเบราว์เซอร์

browser (HTTPS)	fresh ready s	cached ready s	warm median s	warm p95 s	peak private GiB
Edg/157.0.4341.42	6.344	1.265	1.684	1.711	2.075
Chrome/157.0.8103.39	5.516	1.284	1.687	1.694	1.962
Firefox/159.0.1	5.980	1.330	1.772	1.794	2.449

ที่มา: จัดทำโดยผู้จัดทำจากผลการทดสอบระบบผ่าน HTTPS

การทดสอบ HTTPS ใช้ชุด staging-build-v4 เดิม ผ่านการอนุมานเชิงหน้าที่จริง 36 ครั้ง (12 ครั้งต่อเบราว์เซอร์) ด้วยตัวอย่างจาก validation ลำดับ 6, 0, 3 และ 1 ครบสี่คำตัดสินและมี input hashes ตรงกับ Python ผลต่าง probabilities ข้ามเบราว์เซอร์สูงสุด 1.11×10^{-16} เป็นผลของตัวอย่างทดสอบเหล่านี้ ไม่ใช่การประเมิน Macro F1 ของคลังข้อมูลบนเบราว์เซอร์

HTTPS fresh profile หมายถึง browser cache ว่าง แต่ CDN ถูกอุ่นจากการตรวจ asset hash ก่อนแล้ว ไม่ใช่ origin cold start Cached เป็น reload ใน process เดิม ไม่ทดสอบ restart/offline Memory เก็บประมาณทุก 100 ms รวม browser process family; private commit ไม่ใช่ resident RAM และ RSS อาจนับ shared pages ซ้ำ

n=4 ทำให้ p95 เป็นเพียงค่าพรรณนา ไม่ใช่ SLA ระหว่างทดสอบใช้ headless และปิด external fonts จึงยังไม่รับรอง headed visual rendering ตามปกติ

ตารางที่ 4.11 ความสมบูรณ์ ความทนทาน และการแยก staging

รายการ	**ผลที่ตรวจได้บนทั้งสามเบราว์เซอร์**
ไฟล์ HTTPS	16/16 SHA256 ตรง; รวม 161,953,395 ไบต์
ไม่ยืนยันตัวตน	HTML/model/WASM คืน 302 ไป Vercel login ก่อน/หลัง warm cache
runtime	WASM หนึ่งเธรด; ORT 1.26.0-dev.20260416-b7804b056c
แคชและกู้คืน	cached ไม่มี model fetch; corrupt ลบ/โหลดใหม่; cache denied ยังทำงาน
fault recovery	model 503,503,200; init ล้มเหลวแล้วสร้าง Worker ใหม่
feedback/admin	ปิดรวม direct route; Supabase attempts 0; feedback submissions 0

สำหรับการทดสอบ inference-only ชุดเดิม CSP/MIME/cache headers และ worker routes ผ่านการตรวจ แต่ Vercel เพิ่ม Access-Control-Allow-Origin: * จึงไม่อ้างว่า CORS จำกัด origin การป้องกันที่วัดคือ authentication และ same-origin CORP/CSP Edge/Chrome มี worker Fetch.enable diagnostic อย่างละ 7 รายการ ซึ่ง zero-inference probe ยืนยันว่า worker CDP ไม่รองรับ (Network.enable ยังใช้ได้); ไม่ใช่ application error ที่ต้องแก้ build Firefox hosted ไม่มี diagnostic ส่วน normal reviewer SSO, mobile, Safari, low-memory และ offline ยังไม่ยืนยัน Production aliases เดิมและ R2 ไม่ถูกแก้

การตรวจรุ่นรวม security

ภายหลังใช้ข้อมูลสังเคราะห์ทดสอบการเข้าสู่ระบบผู้ดูแล การอนุมัติและปฏิเสธรายงาน การเพิกถอน allowlist และการปฏิเสธสิทธิ์ anonymous

ผู้ใช้ทั่วไปไม่มีสิทธิ์อ่านหรือเขียนตารางโดยตรง Edge Function ตรวจ consent, ป้ายกำกับ ความน่าจะเป็นและข้อความก่อนบันทึก และบังคับ pending พร้อมเพดานรวมแบบ atomic 10 ครั้งต่อนาทีและ 100 ครั้งต่อชั่วโมง การทดสอบ concurrent RPC 12 ธุรกรรมรับ 10 และปฏิเสธ 2 ในหนึ่งนาที เป็นการทดสอบฝั่งบริการ ไม่ใช่การผ่าน CAPTCHA ของมนุษย์ 12 ครั้ง CAPTCHA จริงผ่านในเบราว์เซอร์ปกติของผู้ใช้ แต่ automation บางกรณีไม่ผ่าน จึงไม่อ้างว่าทุกกรณีผ่านหรือเป็นการรับรองความปลอดภัยสากล

บทที่ 5
สรุป

5.1 สรุปผลการดำเนินงาน

การดำเนินงานโครงงานพิเศษระยะที่ 2 บรรลุวัตถุประสงค์ โดยมีผลการดำเนินงานสรุปได้ดังนี้

1. ด้านข้อมูล: ได้ข่าวภาษาไทย 38,428 ตัวอย่างจาก 9,607 กลุ่ม สมดุลสี่ประเภทและแบ่ง gid ประมาณ 80/10/10 ตรวจ lineage และการไม่ทับซ้อนของรหัสกลุ่มได้ แต่ยังมี semantic near-duplicate และตัวแปรกวนที่ไม่ได้ขจัดทั้งหมด

2. ด้านแบบจำลอง: W8A32 ที่เลือกด้วย validation Macro F1 ร้อยละ 88.8342 ได้ test Macro F1 ร้อยละ 87.3705, Accuracy ร้อยละ 87.3569 และ Human Precision ร้อยละ 97.3624 ผ่านเป้าหมายที่ตั้งไว้ โดย Human Recall ร้อยละ 88.3455 ยังต้องสื่อสารความเสี่ยงกล่าวหามนุษย์ผิด

3. ด้านวิศวกรรม: ได้ W8A32 ONNX 106.30 MB พร้อม 400-token parity และ manifest ตรวจ SHA256 รวมทั้งแคชทางเลือกและกู้คืน Worker ผ่านการทดสอบ local/HTTPS เดสก์ท็อปทั้งสามเบราว์เซอร์ รุ่นเผยแพร่ยังใช้ model/tokenizer/runtime เดิม ไม่เขียนทับ R2 และไม่รับประกันต้นทุนหรือหน่วยความจำศูนย์

4. ด้านระบบสนับสนุน: เปิดใช้ข้อมูลป้อนกลับและผู้ดูแลด้วย Supabase Auth, allowlist/RLS, Edge Function และ Turnstile ตรวจความยินยอม ข้อมูลและอัตราการส่งฝั่งบริการ พร้อมทดสอบการอนุญาตและการเปลี่ยนสถานะด้วยข้อมูลสังเคราะห์ ข้อมูลป้อนกลับจริงไม่ใช่ส่วนหนึ่งของการประเมินแบบจำลอง

5. ด้านการวิเคราะห์: บรรยาย GPT magnet effect จากเมทริกซ์และวิเคราะห์กลุ่ม script-mixing เชิงสำรวจ พบความสัมพันธ์กับการแจกแจงข้อมูลหลายด้าน แต่ไม่อ้างเหตุและผล ความเป็นงานแรก หรือความสามารถบน benchmark code-switching ที่มีผู้เชี่ยวชาญกำกับ

เมื่อเทียบกับ SP1 ระยะ SP2 เพิ่มขนาดข้อมูลและขอบเขตสี่ประเภท พร้อมบันทึกการวินิจฉัย quantization, tokenizer parity และทดสอบระบบจริงอย่างตรวจสอบย้อนกลับได้ ผลจำแนกจำกัดอยู่ที่แบบจำลองตรึงและชุดข้อมูลข่าว ผลวัด inference-only staging เป็นหลักฐานระยะก่อนหน้า ส่วนรุ่นเผยแพร่รวม security มีการตรวจการทำงานแยกต่างหาก ไม่ใช่การรับรองความปลอดภัยครบทุกสภาพแวดล้อม

5.2 อภิปรายผล

ผลการศึกษายืนยันความสำเร็จของการจำแนกแหล่งกำเนิดข้อความภาษาไทยแบบสี่ประเภทภายใต้ขอบเขตข้อมูลข่าวที่กำหนด แบบจำลอง W8A32 ที่ตรึงแล้วได้ Macro F1 บนชุดทดสอบร้อยละ 87.3705 สูงกว่าเป้าหมายร้อยละ 85 อยู่ 2.3705 จุดร้อยละ และ Human Precision ร้อยละ 97.3624 สูงกว่าเป้าหมายร้อยละ 95 อยู่ 2.3624 จุดร้อยละ ผลนี้แสดงว่าการปรับแต่งแบบจำลองภาษาไทยร่วมกับการจัดเตรียมข้อมูลและการตรวจสอบท่อส่งการอนุมานสามารถตอบวัตถุประสงค์ของโครงงานได้ โดยต้องตีความผลด้านการระบุค่ายร่วมกับตัวชี้วัดรายประเภทในบทที่ 4

ความสำเร็จเชิงวิศวกรรมอยู่ที่การรักษาผลจำแนกขณะลดขนาดการจัดเก็บและดาวน์โหลดแบบจำลอง W8A32 เก็บน้ำหนักที่เลือกเป็น INT8 แบบรายช่องสัญญาณหรือรายแถวพร้อมสเกล FP32 แล้วคืนค่าน้ำหนักเพื่อคำนวณแบบ FP32 โดยไม่ quantize activations ขนาดไฟล์ลดจาก FP32 ประมาณ 421.21 MB เหลือ 106.30 MB หรือประมาณร้อยละ 74.76 จึงเป็นทางเลือกที่เอื้อต่อการส่งแบบจำลองไปทำงานในเบราว์เซอร์ ผลลดขนาดนี้ไม่เท่ากับการลดหน่วยความจำขณะทำงานหรือเวลาอนุมานในสัดส่วนเดียวกัน

เมื่อเทียบผลบนชุดทดสอบเดียวกัน Macro F1 ลดจาก FP32 ร้อยละ 87.5074 เป็น W8A32 ร้อยละ 87.3705 ต่างกัน 0.1369 จุดร้อยละ โดยคำทำนายเปลี่ยน 20 จาก 3,844 ตัวอย่างและตรงกันร้อยละ 99.4797 จึงบ่งชี้ว่าแบบจำลองที่เลือกคงผลจำแนกโดยรวมไว้ได้ใกล้เคียงรุ่นอ้างอิงในข้อมูลนี้ แต่ไม่ได้แสดงความเท่าเทียมเชิงสถิติหรือรับรองผลในประชากรอื่น การเลือก W8A32 อาศัย validation Macro F1 ร้อยละ 88.8342 ร่วมกับขนาดและความเข้ากันได้ของรันไทม์ มิใช่การเลือกใหม่จากผลทดสอบสุดท้าย

Human Precision ที่สูงหมายถึงข้อความที่ทำนายเป็น Human ส่วนใหญ่เป็นมนุษย์ตามป้ายกำกับของชุดข้อมูล แต่ไม่ใช่การรับรองว่าจะไม่กล่าวหามนุษย์ผิด Human Recall เท่ากับร้อยละ 88.3455 และข้อความมนุษย์ 112 จาก 961 ตัวอย่างถูกจัดเป็น AI คิดเป็นร้อยละ 11.6545 ความแตกต่างระหว่างตัวชี้วัดทั้งสองจึงมีนัยต่อการใช้งานจริง ระบบควรเป็นข้อมูลประกอบการพิจารณาของมนุษย์ ไม่ใช้คำตัดสินหรือคะแนน softmax เป็นหลักฐานความผิดหรือความน่าจะเป็นที่ผ่านการสอบเทียบแล้ว

ผลที่ได้ใช้กับข่าวภาษาไทยเป็นหลักจากแหล่งและผู้สร้างในชุดข้อมูล ไม่อาจสรุปโดยอัตโนมัติถึงเรียงความ สื่อสังคมออนไลน์ บทสนทนา บทความวิชาการ หรือผู้สร้างที่ไม่เคยประเมิน ประเภท Other เป็นกลุ่มผู้สร้างที่กำหนดไว้ ไม่ใช่กลไกตรวจพบผู้สร้างที่ไม่รู้จัก อีกทั้ง Human สืบจากการจัดประเภทแหล่งข่าว มิได้พิสูจน์ว่าผู้เผยแพร่ไม่ใช้ AI ช่วย ส่วนป้าย AI สืบจาก pipeline และ CSV

ไม่ใช่การรับรองผู้ให้บริการรายคำขอ โดยโหมด polish มีข้อความมนุษย์เป็นต้นทาง
นิยามป้ายกำกับจึงเป็นนิยามเชิงปฏิบัติการที่ควรมีหลักฐานแหล่งกำเนิดและความยินยอมรองรับในการศึกษาต่อไป

ในด้านระเบียบวิธี การแบ่งระดับ gid
ช่วยให้ต้นฉบับและฉบับที่สร้างจากบทความรหัสเดียวกันอยู่ในส่วนข้อมูลเดียวกัน
แต่ไม่ขจัดข่าวคล้ายกันเชิงความหมายหรือการปนเปื้อนจาก pretraining ทั้งหมด
ชุดทดสอบเดิมเคยถูกตรวจดูระหว่างการวินิจฉัยรุ่นก่อน แม้การเลือก configuration ของ W8A32 ใช้ validation และการประเมินสุดท้ายใช้แบบจำลองที่ตรึงแล้ว
จึงต้องเรียกผลนี้ว่าการประเมินแบบจำลองตรึงบน original held-out split
ไม่ใช่การทดสอบอิสระที่ไม่เคยตรวจดูมาก่อน

การวิเคราะห์อักษรไทย–
ละตินช่วยขยายความเข้าใจการแจกแจงของข้อมูลและผลจำแนกในกลุ่มย่อย
แต่เป็นหลักฐานเชิงสำรวจ มิใช่ผลบน benchmark การสลับภาษาไทย–
อังกฤษที่กำกับโดยผู้เชี่ยวชาญ
การมีอักษรละตินไม่เท่ากับการใช้คำอังกฤษหรือการสลับภาษาเสมอไป และข้อมูลไม่มี language spans นโยบายจำแนกคำยืมหรือชื่อเฉพาะ และ inter-annotator agreement
ผลต่างระหว่างกลุ่มสัมพันธ์กับแหล่งข่าว ความยาว ประเภท และโหมดสร้าง
จึงไม่พิสูจน์เหตุและผลของ code-switching การประเมินเสริมควรมี provenance
และผู้กำกับข้อมูลสองภาษา โดยไม่ใช้ตัวตรวจจับคัดเฉพาะตัวอย่างที่เอื้อต่อแบบจำลอง

การทดสอบ protected HTTPS Preview
สนับสนุนว่าผลงานจำแนกสามารถเชื่อมกับเว็บแอปพลิเคชันและทำงานในสภาพแวดล้อมเดสก์ท็อปที่ทดสอบได้จริง โดยผ่านการอนุมานเชิงหน้าที่ 36 ครั้งใน Edge, Chrome และ Firefox
รวมทั้งการตรวจแคชและการกู้คืน อย่างไรก็ตาม ผลดังกล่าวเป็นหลักฐานการทำงานของระบบ
ไม่ทดแทนการประเมินความแม่นยำของคลังข้อมูล และไม่ใช่การรับรองความปลอดภัยสากล
การปิด feedback/admin และการไม่พบคำขอ Supabase เป็นขอบเขตของ staging ชุดเดิม
ส่วนรุ่นเผยแพร่ภายหลังเพิ่มการควบคุมสิทธิ์และการตรวจข้อมูลป้อนกลับ
โดยไม่เปลี่ยนผลประเมินแบบจำลอง
ส่วนข้อจำกัดด้านทรัพยากรและสภาพแวดล้อมได้อธิบายไว้ในหัวข้อ 1.5
และแนวทางขยายผลอยู่ในหัวข้อ 5.3

5.3 ข้อเสนอแนะและแนวทางการพัฒนาในอนาคต

1. การตรวจจับแบบเซตเปิด (Open-set Recognition): พัฒนากลไกให้ระบบระบุได้ว่าข้อความมาจากแบบจำลองนอกชุดฝึกสอน แทนการบังคับจัดเข้า 4 ประเภทเสมอ แนวทางที่เป็นไปได้รวมถึงการกำหนดเกณฑ์ขั้นต่ำของความเชื่อมั่น (confidence thresholding) บนการแจกแจง softmax, เทคนิค energy-based out-of-distribution detection หรือการเพิ่มประเภท "ไม่ทราบค่าย" ที่ฝึกด้วยตัวอย่างจากค่ายหลากหลายนอกเป้าหมาย

2. ขยายการประเมินด้วยข้อมูลต่างโดเมนและผู้สร้างที่ไม่เคยฝึก พร้อม untouched holdout ใหม่และ provenance ที่ตรวจสอบได้ ข้อมูล feedback ต้องตรวจป้าย กำกับ ความยินยอมและสิทธิ์ใช้งานก่อนนำไปวิจัย แม้ระบบรับข้อมูลจะควบคุมสิทธิ์แล้ว ควรเก็บ linguistic pilot 480 ข้อความตาม protocol ในอนาคต ไม่ใช่ผลทดลองที่เสร็จแล้ว

3. ศึกษา WebGPU/multi-threaded WASM และความเข้ากันได้มือถือ Safari offline และ low-memory ด้วยเกณฑ์ใหม่ที่วัดจริง ไม่เปลี่ยน runtime หรือ threshold โดยไม่มี regression และ validation แยกต่างหาก

4. การเสริมความปลอดภัยต่อเนื่อง: ประเมิน MFA สำหรับผู้ดูแล การหมดอายุและเพิกถอน session ตลอดจน CAPTCHA กรณีผิด hostname/action และหมดอายุให้ครบถ้วน ปรับมาตรการป้องกัน abuse และติดตามภาระจริงโดยคำนึงถึงความเป็นส่วนตัว การมี Auth/RLS และเพดานอัตราปัจจุบันไม่ใช่หลักประกันต่อภัยคุกคามทุกแบบ

5. การศึกษาความทนทาน (Robustness): ประเมินระบบต่อการโจมตีเชิงปรับแต่งข้อความตามแนวทาง RAID (Dugan et al., 2024) เช่น การถอดความ (paraphrasing), การแทรกอักขระรบกวน และการผสมข้อความมนุษย์–ปัญญาประดิษฐ์ในเอกสารเดียว (mixed authorship) ซึ่งเป็นรูปแบบการหลบเลี่ยงที่พบจริงในบริบทการศึกษา

รายการอ้างอิง

[1] Lowphansirikul, L., Polpanumas, C., Jantrakulchai, N., & Nutanong, S. (2021). WangchanBERTa: Pretraining transformer-based Thai language models. *arXiv preprint arXiv:2101.09635*.

[2] Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., & Finn, C. (2023). DetectGPT: Zero-shot machine-generated text detection using probability curvature. *Proceedings of ICML 2023*. arXiv:2301.11305.

[3] Uchendu, A., Ma, Z., Le, T., Zhang, R., & Lee, D. (2021). TURINGBENCH: A benchmark environment for Turing test in the age of neural text generation. Findings of EMNLP 2021, 2001–2016. https://aclanthology.org/2021.findings-emnlp.172/

[4] Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., et al. (2024). M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection. Proceedings of EACL 2024, 1369–1407. https://aclanthology.org/2024.eacl-long.83/

[5] Dugan, L., Hwang, A., Trhlík, F., Zhu, A., Ludan, J. M., Xu, H., Ippolito, D., & Callison-Burch, C. (2024). RAID: A shared benchmark for robust evaluation of machine-generated text detectors. Proceedings of ACL 2024. https://aclanthology.org/2024.acl-long.674/

[6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.

[7] Martin, L., Muller, B., Ortiz Suárez, P. J., Dupont, Y., Romary, L., de la Clergerie, É., et al. (2020). CamemBERT: A tasty French language model. Proceedings of ACL 2020. https://aclanthology.org/2020.acl-main.645/

[8] Microsoft. ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator. [Online]. Available: https://onnxruntime.ai/

[9] Hugging Face. Optimum: Accelerating training and inference of Transformers. [Online]. Available: https://huggingface.co/docs/optimum

[10] WebAssembly Community Group. WebAssembly SIMD proposal. [Online]. Available: https://github.com/WebAssembly/simd

[11] Cloudflare. R2 Object Storage: Zero egress fee object storage. [Online]. Available: https://www.cloudflare.com/developer-platform/r2/

[12] Vercel. Vercel: Develop. Preview. Ship. [Online]. Available: https://vercel.com/

[13] Supabase. Supabase: The open source Firebase alternative. [Online]. Available: https://supabase.com/

[14] OpenAI. GPT-4o mini: Advancing cost-efficient intelligence. [Online]. Available: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[15] Google. (2025). Gemini 2.5 Flash-Lite is now stable and generally available. [Online]. Available: https://developers.googleblog.com/en/gemini-25-flash-lite-is-now-stable-and-generally-available/

[16] DeepSeek-AI. (2024). DeepSeek-V3 technical report. arXiv preprint arXiv:2412.19437.

[17] Qwen Team, Alibaba Cloud. (2024). Qwen2.5 technical report. arXiv preprint arXiv:2412.15115.

ภาคผนวก

ภาคผนวก ก
การเปิดเผยการใช้งาน Generative AI

ก.1 AI ที่ใช้สร้างชุดข้อมูลสังเคราะห์ (Synthetic Data Generation)

เครื่องมือที่ใช้: Application Programming Interface (API) ของ OpenAI (gpt-4o-mini), Google GenAI (gemini-2.5-flash-lite) และ OpenRouter (deepseek/deepseek-chat, qwen/qwen-2.5-7b-instruct)

ส่วนที่ใช้และวัตถุประสงค์: สร้างข้อมูล AI เป้าหมายก่อนกรอง 30,000 ตัวอย่าง หลังคงกลุ่มครบสี่ประเภทมี AI รวม 28,821 ตัวอย่างใน V3 สุดท้าย ใช้แม่แบบ polish/pure ร่วมกัน แต่ sampling ของ API ไม่เท่ากันทุกค่ายตามที่อธิบายในบทที่ 3

ก.2 AI ด้านการพัฒนาโปรแกรมและการตรวจสอบข้อผิดพลาด (Programming & Debugging)

เครื่องมือที่ใช้: Claude (Anthropic)

ส่วนที่ใช้และวัตถุประสงค์: ใช้ AI ช่วยการพัฒนาตามประวัติเดิม ต่อมาใช้ Codex/ChatGPT ช่วยวินิจฉัย checkpoint/quantization แก้ tokenizer parity และ reliability จัดทำสคริปต์ตรวจสอบ และสรุปหลักฐานที่เก็บไว้ ผลเชิงตัวเลขมาจาก artifact/การทดสอบที่บันทึก ไม่ใช่ตัวเลขที่ AI แต่งขึ้น การแก้รายงานฉบับนี้ใช้ AI ช่วยร่างและตรวจความสอดคล้อง ผู้จัดทำต้องตรวจและรับผิดชอบก่อนส่ง

ก.3 AI ด้านการสืบค้นวรรณกรรมและการจัดเตรียมเอกสาร (Literature Search & Documentation)

เครื่องมือที่ใช้: Gemini (Google)

ส่วนที่ใช้และวัตถุประสงค์: ใช้สนับสนุนการค้นและจัดรูปแบบวรรณกรรมตามประวัติเดิม การทบทวนรายงานใช้แหล่งเจ้าของผลงานตรวจ metadata บางรายการ ไม่ถือว่าข้อความที่ AI เสนอเป็น citation ที่ยืนยันแล้ว ฉบับนี้ถอดรายการที่ตรวจสอบแหล่งตีพิมพ์ไม่ได้ออก และจัดทำภาพที่ 2.1 ใหม่จากกระบวนการทดลองของโครงงาน ผู้จัดทำยังต้องทบทวนความถูกต้องของการอ้างอิงและรับผิดชอบเนื้อหาก่อนส่ง

ก.4 คำรับรองความรับผิดชอบต่อผลงาน

ผู้จัดทำยืนยันว่าการประยุกต์ใช้เครื่องมือ Generative AI ข้างต้นเป็นไปเพื่อสนับสนุนกระบวนการเรียนรู้และการพัฒนาชิ้นงานเท่านั้น

ผู้จัดทำมีความเข้าใจในหลักการทำงาน แนวคิด สถาปัตยกรรมระบบ โครงสร้างคลังข้อมูล อัลกอริทึม ตลอดจนซอร์สโค้ดและข้อความทั้งหมดที่ปรากฏในโครงงานนี้อย่างครบถ้วน

ผลหลักของ W8A32 คือ test Macro F1 ร้อยละ 87.3705 และ Human Precision ร้อยละ 97.3624 แยกจาก FP32 reference และ validation ตามบทที่ 4 หลักฐาน logits, metrics และ hash ถูกเก็บในไดเรกทอรีทดลองที่อ้างถึง

การจัดทำรายงานอ้างการประเมินและหลักฐานจริงที่บันทึกไว้โดยใช้เครื่องมืออัตโนมัติช่วยตรวจสอบ ผู้จัดทำต้องทบทวนความถูกต้องของผล การตีความ และการอ้างอิงด้วยตนเองก่อนรับรองเอกสาร

ผู้จัดทำเป็นผู้รับผิดชอบต่อความถูกต้องทางวิชาการ ความสมบูรณ์ และผลสัมฤทธิ์ของเอกสารและโครงงานชิ้นนี้แต่เพียงผู้เดียว

ภาคผนวก ข
สคริปต์การจัดการชุดข้อมูลและการแบ่งข้อมูลระดับกลุ่ม (Data Pipeline & Splitting)

ข.1 สคริปต์ทำความสะอาดและคัดกรองข้อมูล (Data Cleaning Filters)

โค้ดดังกล่าวอยู่ในไฟล์ scripts/build_dataset_v3.py ซึ่งเป็น Pipeline สำหรับเตรียมและทำความสะอาดชุดข้อมูล V3 โดยมีรายละเอียดโค้ดแต่ละส่วนดังนี้:

1. การตัดแต่ง Markdown และลบข้อความเกริ่นนำสำเร็จรูป (Scaffolding)

ส่วนนี้ใช้ Regular Expression ในการตัด Markdown ทั่วไป (**, *, `, #) รวมถึงคำขึ้นต้นสำเร็จรูป (Preamble เช่น "ได้เลย...", "Here is...") และป้ายกำกับส่วนของเนื้อหา (Scaffold Labels เช่น "คำนำ:", "เนื้อข่าว:") ที่โมเดลขนาดเล็กมักสร้างติดมา

```
MARKDOWN_PATTERNS = [
    (re.compile(r"\*\*"), ""),
    (re.compile(r"\*"), ""),
    (re.compile(r"`"), ""),
    (re.compile(r"^\s*#{1,6}\s*", re.MULTILINE), ""),
    (re.compile(r"#{1,6}"), ""),
]
# Small local models (label 3) sometimes emit meta scaffolding despite the
# prompt ("คำนำ:", "เนื้อข่าว:", "Here is ...:"). Strip it so it can't become a
# formatting tell. Harmless on clean GPT/Gemini text (rarely matches).
_SCAFFOLD_LABELS = [
    "คำตอบทั้งหมด", "คำตอบ", "คำนำ", "เนื้อข่าว", "เนื้อหาข่าว", "บทความ",
    "หัวข้อข่าว", "หัวข้อ", "สรุปข่าว", "บทสรุป", "สรุป", "ข่าว",
]

SCAFFOLD_LABEL_RE = re.compile(
    r"(?:^|\n)\s*(?:" + "|".join(_SCAFFOLD_LABELS) + r")\s*[: ： ]\s*", re.M)
SCAFFOLD_PREAMBLE_RE = re.compile(
    r"^\s*(?:sure|here(?:'s| is)|certainly|ได้เลย|นี่คือ)[^\n]*\n+", re.I)
```

```
def strip_markdown(text: str) -> str:
    if not isinstance(text, str):
        return ""
    text = SCAFFOLD_PREAMBLE_RE.sub("", text)
    text = SCAFFOLD_LABEL_RE.sub(" ", text)
    for pat, repl in MARKDOWN_PATTERNS:
        text = pat.sub(repl, text)
    return re.sub(r"\s+", " ", text).strip()
```

2. ฟังก์ชันตรวจนับสัดส่วนอักขระภาษาไทย (Thai Character Ratio)

ฟังก์ชันนี้ตรวจนับจำนวนอักขระที่อยู่ในช่วง Unicode ภาษาไทย (\u0E00 ถึง \u0E7F หรือในโค้ดเขียนเป็น "\u0E00" ถึง "\u0E7F") เทียบกับความยาวทั้งหมดของข้อความ เพื่อคัดกรองข้อความที่เป็นภาษาอังกฤษล้วนหรือภาษาผสมที่โมเดลตอบผิดคำสั่งทิ้ง

```
MIN_THAI_RATIO = 0.5   # drop English/mixed degenerate generations
def thai_ratio(text: str) -> float:
    """Fraction of Thai-script chars. Filters degenerate English/mixed outputs
    (small models sometimes ignore the Thai instruction) so the classifier
    can't learn a language tell instead of writing style."""
    t = str(text)
    if not t:
        return 0.0
    thai = sum(1 for ch in t if "\u0E00" <= ch <= "\u0E7F")  # ช่วง Unicode 0x0E00 ถึง 0x0E7F
    return thai / len(t)
```

3. จุดที่นำมารวมกันในกระบวนการ Clean Dataset

ฟังก์ชัน clean จะเรียกใช้ทั้งการตัดแต่ง Markdown/Scaffolding และการกรองสัดส่วนภาษาไทย ≥ 0.5 ร่วมกัน:

```
def clean(df: pd.DataFrame) -> pd.DataFrame:
    df = df.dropna(subset=["text"]).copy()
    df["text"] = df["text"].astype(str).map(strip_markdown)
    df = df[(df["text"].str.len() >= MIN_LEN)]
    df = df[df["text"].map(thai_ratio) >= MIN_THAI_RATIO]  # กรองเฉพาะสัดส่วนภาษาไทย >= 0.5
    df["text"] = df["text"].str.slice(0, MAX_LEN)
    df = df.drop_duplicates(subset=["text"])
    df["label"] = df["label"].astype(int)
    return df
```

ข.2 สคริปต์การแบ่งข้อมูลระดับกลุ่มบทความ (Group-level Splitting)

โค้ดส่วนนี้อยู่ในไฟล์ scripts/build_dataset_v3.py โดยแบ่งออกเป็นส่วนการสับ/จัดสรร และส่วนคำสั่ง Assertion ดังนี้:

1. ค่าคงที่กำหนดสัดส่วน (80:10:10)

```
RANDOM_STATE = 42
TRAIN_FRAC = 0.8     # 80% Train
VAL_FRAC = 0.1       # 10% Validation (และที่เหลืออีก 10% จะเป็น Test)
```

2. โค้ดสับ (Shuffle) และจัดสรรรหัสกลุ่มบทความ (gid) ในสัดส่วน 80:10:10

กระบวนการนี้จะทำในระดับ gid (Group ID ของบทความต้นทาง) แทนที่จะสับระดับแถว เพื่อป้องกันไม่ให้สำนวน/ข้อความที่แปลงมาจากบทความเดียวกันหลุดข้ามชุดข้อมูล:

```
complete_list = sorted(complete)
    # สับลำดับ gid แบบกำหนด random_state คงที่
    rng = pd.Series(complete_list).sample(frac=1.0, random_state=RANDOM_STATE).tolist()
    if args.max_groups and args.max_groups < len(rng):
        rng = rng[: args.max_groups]
        complete = set(rng)
        print(f"Capped to {len(rng)} articles ({len(rng)} rows/class).")
    # Group-aware 80/10/10 split on the unique gids.
    n = len(rng)
    train_end = int(TRAIN_FRAC * n)
```

```
    val_end = int((TRAIN_FRAC + VAL_FRAC) * n)


    split_of = {}
    for g in rng[:train_end]:
        split_of[g] = "train"        # 0% ถึง 80%
    for g in rng[train_end:val_end]:
        split_of[g] = "val"          # 80% ถึง 90%
    for g in rng[val_end:]:
        split_of[g] = "test"         # 90% ถึง 100%
 # Assemble rows, filtered to complete gids, tagged by split.
    frames = []
    for lbl, p in parts.items():
        p = p[p["gid"].isin(complete)].copy()
        p["split"] = p["gid"].map(split_of)
        frames.append(p)
    allrows = pd.concat(frames, ignore_index=True)
```

3. คำสั่งตรวจสอบการไม่ข้ามกลุ่มข้อมูล (Post-hoc Leakage Assertion)

หลังจากประกอบทุกชุดข้อมูลเข้าด้วยกัน จะมีคำสั่งตรวจสอบว่าแต่ละ gid ต้องปรากฏอยู่ใน Split เดียวเท่านั้น (nunique() == 1):

```
# Leakage assertion: no gid spans two splits (guaranteed by construction).
    spans = allrows.groupby("gid")["split"].nunique()
    assert (spans == 1).all(), "LEAKAGE: a gid appears in multiple splits!"
    print("\nLeakage check passed: every article confined to a single split.")
```

ข.3 คำสั่งกำเนิดข้อความสังเคราะห์ (Prompt Templates)

แม่แบบคำสั่งอยู่ใน scripts/gen_prompts.py และใช้ build_prompt ร่วมกันในสคริปต์ผู้สร้าง เป็นการใช้แม่แบบกลาง มิใช่หลักฐานว่า provider request, sampling หรือคำตอบซ้ำกันทุกไบต์

1. โหมด Polish (เรียบเรียงและขัดเกลาเนื้อข่าวต้นฉบับ)

ใช้สำหรับสร้างข้อมูลโดยให้โมเดลได้รับเนื้อความเต็มของมนุษย์ แล้วขัดเกลาสำนวนใหม่

```
POLISHED_PROMPT_TMPL = """คุณคือบรรณาธิการข่าวมืออาชีพระดับสำนักข่าวชั้นนำของไทย
ภารกิจ: เรียบเรียงและขัดเกลาข่าวต่อไปนี้ใหม่ ให้กระชับ อ่านง่าย และมีโทนเป็นทางการมากขึ้น

ข้อกำหนด:
- คงข้อเท็จจริง ตัวเลข วันเวลา ชื่อบุคคล ชื่อสถานที่ และรายละเอียดสำคัญให้ตรงกับต้นฉบับทุกประการ
- ห้ามเพิ่มข้อมูลใหม่ที่ไม่ได้อยู่ในต้นฉบับ
- ใช้ภาษาไทยที่เป็นธรรมชาติของผู้สื่อข่าวมืออาชีพ
- ห้ามใส่คำนำหน้า เช่น "นี่คือข่าวที่เรียบเรียงใหม่" หรือคำอธิบายใดๆ ส่งคืนเฉพาะเนื้อข่าวที่ขัดเกลาแล้ว
- ห้ามใช้สัญลักษณ์ Markdown ในการจัดรูปแบบข้อความเด็ดขาด (เช่น ห้ามใช้ ** หรือ *)

ต้นฉบับ:
\"\"\"
{human_text}
\"\"\"
"""
```

2. โหมด Pure (เขียนบทความข่าวขึ้นใหม่จากหัวข้อข่าว)

ใช้สำหรับสร้างข้อมูลโดยให้โมเดลได้รับเพียงหัวข้อข่าว (Headline) และแต่งเนื้อข่าวขึ้นมาใหม่ทั้งหมด

```
PURE_PROMPT_TMPL = """คุณคือผู้สื่อข่าวมืออาชีพชาวไทย
ภารกิจ: เขียนบทความข่าวขึ้นใหม่ทั้งหมดเป็นภาษาไทย โดยใช้เพียงหัวข้อข่าวด้านล่างเป็นแรงบันดาลใจ

ข้อกำหนด:
- เขียนเป็นภาษาไทยธรรมชาติแบบสำนักข่าว
- โครงสร้างแบบข่าว: ย่อหน้าเปิด สรุปประเด็นหลัก รายละเอียดเพิ่มเติม และย่อหน้าปิด
- ความยาวประมาณ 200-400 คำ
- ห้ามใส่หัวข้อซ้ำ ห้ามใส่คำนำหน้าหรือคำอธิบายนอกตัวบทข่าว ส่งคืนเฉพาะเนื้อข่าว
- ห้ามใช้สัญลักษณ์ Markdown ในการจัดรูปแบบข้อความเด็ดขาด (เช่น ห้ามใช้ ** หรือ *)

หัวข้อข่าว: {title}
"""
```

3. พารามิเตอร์ควบคุมความเสมอภาค (Parity Hyperparameters)

ค่าคงที่ GENERATION_TEMPERATURE = 0.8 ปรากฏในโมดูลกลาง แต่การส่งผ่านต้องตรวจที่ API call: OpenAI/OpenRouter ระบุค่าและขีดจำกัด 900 tokens ส่วน Gemini ไม่ส่ง generation config ในสคริปต์ที่ตรวจ จึงไม่อ้างว่าทุกค่ายใช้ sampling เหมือนกัน โค้ดด้านล่างแสดงเฉพาะแม่แบบและค่าคงที่ ไม่ใช่บันทึก API ทุกคำขอ

```
GENERATION_TEMPERATURE = 0.8
def build_prompt(mode: str, title: str, human_text: str) -> str:
  """Return the filled prompt for the given mode ('polish' or 'pure')."""
  if mode == "polish":
    return POLISHED_PROMPT_TMPL.format(human_text=human_text)
  if mode == "pure":
    return PURE_PROMPT_TMPL.format(title=title)
  raise ValueError(f"unknown mode: {mode!r} (expected 'polish' or 'pure')")
```

ภาคผนวก ค
สคริปต์การแปลงและบีบอัดแบบจำลอง (Model Optimization & Quantization)

ค.1 สคริปต์การส่งออกกราฟการคำนวณ (ONNX Export)

คำสั่งส่งออกแบบจำลอง WangchanBERTa ไปเป็นรูปแบบ ONNX อยู่ในไฟล์ scripts/export_onnx.py โดยใช้ไลบรารี Hugging Face Optimum (optimum.onnxruntime) มีรายละเอียดโค้ดดังนี้:

```
from optimum.onnxruntime import ORTModelForSequenceClassification
from transformers import AutoTokenizer
import os
import shutil
# กำหนด Path ของโมเดลที่ Fine-tuned เสร็จแล้ว และโฟลเดอร์ปลายทาง
MODEL_DIR = os.path.join(ROOT_DIR, "model_output")
ONNX_DIR = os.path.join(SCRIPT_DIR, "onnx_model")
# โหลด Tokenizer (จากโฟลเดอร์โลคอล หรือดึง fallback จาก HF Hub)
try:
    tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
except Exception:
    tokenizer = AutoTokenizer.from_pretrained("airesearch/wangchanberta-base-att-spm-uncased")
# คำสั่งแปลงและส่งออกเป็น ONNX ด้วย optimum (กำหนด export=True)
ort_model = ORTModelForSequenceClassification.from_pretrained(MODEL_DIR, export=True)
# บันทึกโมเดล ONNX และ Tokenizer ไปยังไดเรกทอรีเป้าหมาย
if os.path.isdir(ONNX_DIR):
    shutil.rmtree(ONNX_DIR, ignore_errors=True)
ort_model.save_pretrained(ONNX_DIR)
tokenizer.save_pretrained(ONNX_DIR)
onnx_file = os.path.join(ONNX_DIR, "model.onnx")
print(f"ONNX saved ⟶ {onnx_file}  ({os.path.getsize(onnx_file)/1e6:.1f} MB)")
```

ผลลัพธ์ของขั้นตอนนี้: ได้ไฟล์โมเดล ONNX รูปแบบเต็ม (FP32) ขนาดประมาณ 421.2 MB

ค.2 ประวัติสคริปต์ Dynamic INT8 ที่ไม่ใช่แบบจำลองสุดท้าย

โค้ด scripts/export_onnx.py ต่อไปนี้เป็นประวัติ per-tensor dynamic INT8 เดิม มิใช่ขั้นตอนผลิต artifact W8A32 ที่ใช้ในระบบฉบับสมบูรณ์ ห้ามใช้เป็นหลักฐานว่าการบีบอัดเดิมรักษาความแม่นยำ หรือรันเพื่อเขียนทับแบบจำลองที่อนุมัติ

```
from optimum.onnxruntime import ORTQuantizer
from optimum.onnxruntime.configuration import AutoQuantizationConfig
# 1. สร้าง Quantizer โดยอ้างอิงจากโมเดล ONNX (FP32) เดิม
quantizer = ORTQuantizer.from_pretrained(ONNX_DIR)
# 2. ตั้งค่า Configuration สำหรับ Dynamic Quantization (AVX2)
qconfig = AutoQuantizationConfig.avx2(is_static=False, per_channel=False)
# 3. สั่งประมวลผลการบีบอัดและบันทึกผลลัพธ์
if os.path.isdir(ONNX_QUANT_DIR):
    shutil.rmtree(ONNX_QUANT_DIR, ignore_errors=True)
quantizer.quantize(save_dir=ONNX_QUANT_DIR, quantization_config=qconfig)
tokenizer.save_pretrained(ONNX_QUANT_DIR)
```

ไฟล์เดิมมีขนาด 106,066,136 ไบต์ แต่บนอินพุต Python เดียวกันได้ test Macro F1 ร้อยละ 75.8756 เทียบ FP32 ร้อยละ 87.5074 จึงไม่ใช่รุ่นที่ใช้ในผลสุดท้าย รุ่น W8A32 และขั้นตอน reproducibility แสดงใน ค.3; test diagnostic นี้เกิดก่อนการเลือก configuration ด้วย validation และเปิดเผยเป็นข้อจำกัดแล้ว

ค.3 แบบจำลอง W8A32 ที่เลือกและหลักฐานทำซ้ำ

แบบจำลองสุดท้ายอยู่ที่ experiment/archive-20261108-readiness-7c21/historical/quantization/models/wo_pc_foldable.onnx มี SHA256 4baa5cb331f6cad7d90492d33c3e9a5bf53a45029ca5a563aaa86f7a46bc84a2 ขนาด 106,299,948 ไบต์ เก็บ linear weights แบบ QInt8 [−127,127] per-output-channel จำนวน 74 tensors และ word/position embeddings per-row รวม quantized 76 tensors Scale = max(abs(weight))/127; all-zero scale ใช้ 1, ไม่มี activation quantization; token-type embeddings/bias/normalization คง FP32 ใช้ Cast/Mul foldable คืนค่าน้ำหนัก มิได้รัน dynamic quantizer เดิมซ้ำ

Selection protocol และ candidate manifests อยู่ใน historical/quantization/ โดยใช้ val.csv และระบุ testSetUsed=false ผลตรึงสุดท้ายอยู่ใน experiment/research-evaluation-20261108-w8a32-test-v1/ ได้แก่ protocol-locked.json, selection-

provenance.json, final-metrics.json, confusion-matrix.csv, predictions.jsonl และ comparison-vs-fp32.json ตัวตัดโทเคน SHA256 ea8b168aba1b6e7d34a764a8aedc3ed6991e43029f83bfe463ab7725698e03eb; config mapping ต้องตรง [Human,GPT,Gemini,Other]

Dataset SHA256: train 31adc71c0bbf4cfe8f81600c180c8af80c688fe78da0f44c47811b35f72afb2c; validation 78c834a1479de83d8d18636c4df37775578379dc350fbfdf85084c25198b8989; test 204aa985e146f22f6d44e8e26939d7af620dc2f5e9e5a3140675912f5c9d222a Python 3.11.9, transformers 4.57.6, tokenizers 0.22.2, ONNX Runtime 1.24.3, scikit-learn 1.7.2 เป็นสภาพแวดล้อมประเมินซ้ำที่บันทึกไว้ ไม่อ้างว่าเป็น historical environment ทุก package โดยแน่นอน

การทำซ้ำต้องใช้ไดเรกทอรีใหม่ ตรวจ artifact/hash และ frozen input ก่อน ไม่รันสคริปต์ประวัติที่ล้างปลายทางหรือเขียนทับหลักฐาน ผู้จัดทำเก็บรายงานการทดลอง quantization การตรวจความพร้อมใช้งาน การประเมินแบบจำลองตรึง และการวิเคราะห์ script-mixing แยกเป็นหลักฐานประกอบโครงงาน โดยพิจารณาเงื่อนไขของแต่ละการทดลอง ไม่ใช้ผลต่างชุดแทนกัน

ภาคผนวก ง
สคริปต์ระบบสนับสนุนและระบบอัตโนมัติ (System Infrastructure & Automation)

ง.1 เวิร์กโฟลว์ GitHub Actions สำหรับรักษาสถานะฐานข้อมูล

โค้ดต่อไปนี้เป็น workflow เก่าที่เก็บเพื่อประวัติ ห้ามใช้กับฐานข้อมูลปัจจุบันซึ่งไม่อนุญาต anonymous SELECT ใน user_reports แล้ว workflow ใน source ปัจจุบันเลิกอ่านรายงานดังกล่าว การแสดงตัวอย่างเก่านี้ไม่ใช่ขั้นตอนติดตั้งหรือการรับรองว่าจะป้องกันการพักบริการได้

```
- name: Query user_reports (real DB compute)
env:
SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
SUPABASE_ANON_KEY: ${{ secrets.SUPABASE_ANON_KEY }}
run: |
set -euo pipefail
# Real SELECT against Postgres via PostgREST: SELECT id FROM user_reports LIMIT 1.
# Capture body + HTTP status separately so we can fail loudly if the query
# does NOT actually execute (RLS deny / auth error / paused project) — a plain
# curl exits 0 on those and masks the failure as a green check.
http_code=$(curl -sS -w '%{http_code}' -o response.json \
"${SUPABASE_URL}/rest/v1/user_reports?select=id&limit=1" \
-H "apikey: ${SUPABASE_ANON_KEY}" \
-H "Authorization: Bearer ${SUPABASE_ANON_KEY}" \
-H "Accept: application/json")
echo "HTTP status: ${http_code}"
echo "Response body: $(cat response.json)"
# PostgREST returns 200 with a JSON array for a successful SELECT.
if [ "${http_code}" != "200" ]; then
echo "::error::Keep-alive query did not return 200 — DB compute may not have registered."
exit 1
fi
# Body must be a JSON array (real query result), not an error object.
```

```
if ! grep -q '^\[' response.json; then
echo "::error::Unexpected response shape — expected a JSON array from SELECT."
exit 1
fi
echo "Keep-alive query succeeded; Postgres compute registered."
```

ง.2 หลักฐาน staging และขอบเขตที่ยังไม่รับรอง

หลักฐานการตรวจสอบแยกตามระยะ: staging-build-v4 แบบ inference-only มี 16 ไฟล์และไม่เปลี่ยน production ในขณะนั้น ต่อมารุ่นรวม security มี 18 ไฟล์ที่ตรวจ hash ผ่านการทดสอบ browser และ Admin/Feedback ก่อนเผยแพร่ การเตรียมไฟล์ใช้ lossless reconstruction ไม่ใช่ฝึกหรือ export น้ำหนักใหม่ ผล native, validation, final test, browser fixtures และความเร็ว local/HTTPS เป็นคนละเงื่อนไข ไม่รวมเป็นผลประเมินเดียว

ข้อพิจารณาก่อนส่งรายงาน: ผู้จัดทำต้องทบทวนการอ้างอิงและภาพประกอบ
ตรวจวันที่สอบและลายเซ็นตามหลักฐานมหาวิทยาลัยโดยไม่เติมวันที่สมมติ
ยืนยันรูปแบบเลขหน้าเบื้องต้นกับอาจารย์ที่ปรึกษา
และตรวจสารบัญกับคำอธิบายตารางจากไฟล์ฉบับแก้ไข
เก็บรายงานเดิมและหลักฐานทุกฉบับไว้ ไม่เผยแพร่ browser profiles, token, credentials หรือ URL bypass